



Trabalho de Conclusão de Curso

Controle determinístico para CNC baseado em STM32L475 com DDA de alta frequência

de Valdir Dias Silva Junior

orientado por

Prof. Dr. Ícaro Bezerra Queiroz de Araújo

Universidade Federal de Alagoas
Instituto de Computação
Maceió, Alagoas
26 de Novembro de 2025

UNIVERSIDADE FEDERAL DE ALAGOAS
Instituto de Computação

CONTROLE DETERMINÍSTICO PARA CNC BASEADO EM STM32L475 COM DDA DE ALTA FREQUÊNCIA

Trabalho de Conclusão de Curso submetido
ao Instituto de Computação da Universidade
Federal de Alagoas como requisito parcial
para a obtenção do grau de Engenheiro de
Computação.

Valdir Dias Silva Junior

Orientador: Prof. Dr. Ícaro Bezerra Queiroz de Araújo

Banca Avaliadora:

Ícaro Bezerra Queiroz de Araújo	Prof. Dr., IC-UFAL
Glauber Rodrigues Leite	Prof. Dr., IC-UFAL
Andressa Martins Oliveira	M.e., IC-UFAL

Maceió, Alagoas
26 de Novembro de 2025

Dados para elaboração da ficha catalográfica devem ser inseridos aqui.
Substitua este texto pelo conteúdo oficial fornecido pela biblioteca ou órgão responsável.

Dedicatória

Dedico este trabalho, antes de tudo, à minha família, em especial aos meus pais, Valdir e Maria, que sempre acreditaram em mim e no poder dos estudos mesmo quando o caminho parecia distante. Cada palavra desta monografia carrega um pouco do esforço, do exemplo e do cuidado de vocês ao longo de toda a minha vida.

Dedico também a todos que fizeram parte da minha formação acadêmica, professores e colegas, que contribuíram com conhecimento, questionamentos, incentivos e até cobranças nas horas certas. Sem esse ambiente de aprendizado, troca e desafio constante, este trabalho não teria encontrado o espaço necessário para nascer e amadurecer.

Por fim, dedico a você, leitor, que se interessou por este trabalho e, de alguma forma, compartilha a curiosidade pelo mundo da robótica, do controle e dos sistemas embarcados. Espero que as ideias aqui apresentadas possam inspirar novos projetos, estudos e sonhos, assim como este tema me inspirou ao longo dessa jornada.

Agradecimentos

Em primeiro lugar, agradeço à minha família, base e alicerce de toda a minha trajetória. Aos meus pais, Valdir e Maria, pelo amor, incentivo e apoio incondicionais desde os meus primeiros dias de vida. Cada conquista minha carrega o esforço, o cuidado e os valores que vocês me ensinaram e que foram fundamentais para que eu chegasse até aqui.

À Dona Benedita (Benta), que cuidou de mim com tanto carinho e dedicação, levando-me e buscando-me no colégio quando criança, sempre com alegria e bom humor.

Aos meus tios, tias e primos, agradeço pelo apoio, pelas palavras de motivação e pela confiança em meu potencial. A presença de vocês, seja nos encontros em família, nas conversas ou nas orações, foi um combustível essencial ao longo dessa caminhada.

Aos meus amigos Willieny, Hugo, Eric e a todos os outros que fizeram parte dessa jornada, agradeço pela amizade, pelo companheirismo e pela compreensão nos momentos de cansaço e ausência. Vocês tornaram o percurso mais leve, trazendo apoio nos períodos mais difíceis e alegria nas horas de descontração.

Estendo ainda minha gratidão aos professores, orientadores e colegas de curso que contribuíram, direta ou indiretamente, para minha formação acadêmica e pessoal, compartilhando conhecimento, oportunidades e conselhos valiosos.

A realização deste trabalho só foi possível graças ao apoio e à contribuição de todas essas pessoas, muitas das quais não consigo mencionar nominalmente. A todos, deixo registrada a minha mais profunda gratidão e o meu mais sincero muito obrigado.

Epígrafe

“Your visions will become clear only when you can look into your own heart. Who looks outside, dreams; who looks inside, awakes.”

— Carl Jung

Resumo

Este trabalho apresenta o desenvolvimento de um controlador CNC embarcado voltado a movimentos previsíveis e repetíveis em três eixos. A solução combina um microcontrolador STM32L475, responsável pelas tarefas de tempo real, com uma Raspberry Pi usada como interface de supervisão e envio de comandos. A proposta busca reduzir a dependência de um computador de propósito geral na geração direta dos sinais de movimento, mantendo a flexibilidade de uma interface de alto nível.

O sistema foi construído a partir de uma arquitetura incremental: primeiro foram validados os temporizadores, depois a geração de pulsos para os motores, a leitura dos encoders, o controle em malha fechada e, por fim, a comunicação com o host. A fundamentação discute os conceitos necessários para essa integração, incluindo sistemas CNC, motores de passo, controle PID, comunicação embarcada e geração digital de passos. Os ensaios de bancada indicaram estabilidade na geração de movimento, baixa variação temporal nos serviços críticos e funcionamento consistente do protocolo de comunicação. Como resultado, o trabalho consolida uma base de firmware e documentação para evolução futura do controlador, especialmente em pontos como otimização da comunicação, melhoria da telemetria e ajuste fino dos perfis de movimento.

Palavras-chave: controle CNC; STM32L475; DDA; controlador PID; SPI determinístico.

Abstract

This dissertation presents the development of an embedded CNC controller aimed at predictable and repeatable motion over three axes. The solution combines an STM32L475 microcontroller, responsible for real-time tasks, with a Raspberry Pi used as the supervision interface and command host. The proposal reduces the dependence on a general-purpose computer for direct motion-signal generation while preserving the flexibility of a high-level interface.

The system was built through an incremental architecture: timers were validated first, followed by motor pulse generation, encoder reading, closed-loop control, and host communication. The theoretical background discusses the concepts required for this integration, including CNC systems, stepper motors, PID control, embedded communication, and digital step generation. Bench tests indicated stable motion generation, low temporal variation in critical services, and consistent behavior of the communication protocol. As a result, this work consolidates a firmware and documentation base for future evolution of the controller, especially in communication optimization, telemetry improvements, and fine tuning of motion profiles.

Keywords: CNC control; STM32L475; DDA; PID controller; deterministic SPI.

Lista de Figuras

3.1	Diagrama em blocos da arquitetura de software e controle de movimento. .	19
3.2	Fluxo resumido do comando <code>cnc-cli</code> , desde a validação da sequência até o monitoramento e encerramento da execução.	19
3.3	Sobreposição entre velocidade medida e resposta FOPDT para o eixo X. .	22
3.4	Sobreposição entre velocidade medida e resposta FOPDT para o eixo Y. .	22
3.5	Sobreposição entre velocidade medida e resposta FOPDT para o eixo Z. . .	22
3.6	Modelo 3D do controlador CNC com placa STM32, interface para Raspberry Pi, base de prototipagem e módulos de potência TMC5160.	23
3.7	Modelo 3D do suporte do encoder TMCS-28 e do ponteiro acoplado ao eixo do motor de passo.	24
3.8	Vista dos motores de passo com suportes impressos em 3D, ponteiros e encoder TMCS-28 acoplado ao eixo para realimentação de posição.	25
3.9	Protótipo do controlador CNC montado em bancada, com fonte de alimentação, motores de passo, drivers TMC5160, placa STM32 e Raspberry Pi.	26
3.10	Diagrama em blocos do subsistema de controle de movimento, destacando a separação entre o laço de controle a 1 kHz e o gerador de pulsos a 50 kHz. .	26
3.11	Comparação visual entre o pulso de <code>STEP</code> implementado no firmware e o pulso mínimo requerido pelo TMC5160.	27
3.12	Simulação da operação interna do DDA para um movimento de 7 passos em X e 4 em Y.	28
3.13	Perfil de velocidade da rampa trapezoidal simulado a partir dos parâmetros do firmware.	31
3.14	Cenário de simulação com atrito programável por eixo, destacando janelas de carga e impacto sobre posição, velocidade e erro de sincronismo.	35
4.1	Comparação entre velocidade de passo medida e simulada para o ensaio de seis voltas completas, por eixo.	39

Lista de Tabelas

2.1	Layout do quadro SPI (tamanho fixo: 42 bytes).	8
2.2	Requisição LED (CMD=0x10).	9
2.3	Resposta LED.	9
2.4	Requisição Movimento (CMD=0x20).	10
2.5	Resposta Movimento.	10
2.6	Quadro de escrita TMC5160 (5 bytes, big-endian).	11
2.7	Quadro de leitura TMC5160 (5 bytes, big-endian; resposta no ciclo seguinte).	11
2.8	Exemplos de acessos SPI ao TMC5160.	12
2.9	Estimativa da corrente mínima prática (I_{RMS}).	14
2.10	Registros STM32 L4 utilizados neste trabalho (ver [STMicroelectronics, 2013, STMicroelectronics, 2018a]).	16
2.11	Registros TMC5160 utilizados neste trabalho (ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), sd]).	17
3.1	Comparativo entre tempos mínimos do TMC5160 e tempos configurados no firmware.	27
3.2	Parâmetros de tempo e limites usados no firmware.	33

Lista de Símbolos

- ARR Valor do registrador de auto-reload usado na configuração dos temporizadores.
- α Parâmetro de filtragem exponencial usado no termo derivativo discreto e fator de segurança nas estimativas de corrente mínima.
- B Coeficiente de atrito viscoso equivalente do eixo.
- c Contagem acumulada do encoder.
- D Duty cycle do sinal STEP, isto é, a fração do período em que o pulso permanece em nível alto.
- $\Delta c_k, \Delta p_k, \Delta t_k$ Variações discretas de contagem e tempo usadas na estimação experimental de velocidades.
- $e[k]$ Erro discreto entre referência e medição na amostra k .
- $e_{\text{sync},i}$ Erro de sincronismo do eixo i em relação ao eixo mestre.
- $e_f(t)$ Força contraeletromotriz induzida na fase do motor.
- e_{th} Limiar de sincronismo usado na modulação cruzada de velocidade entre eixos.
- f_{sys} Frequência do clock principal do microcontrolador.
- f_{bus} Frequência do barramento que alimenta o temporizador.
- f_{TIM6} Frequência de interrupção do temporizador TIM6 usada pelo DDA.
- f_{loop} Frequência do laço de controle executado no TIM7.
- $H(t)$ Função degrau unitário de Heaviside.
- I_{RMS} Corrente eficaz de fase adotada no ajuste dos drivers de motor.
- $I_{\text{HOLD}}, I_{\text{RUN}}$ Correntes de sustentação e de regime configuradas no TMC5160.
- k Índice discreto de amostragem.

- N_{steps} Número de pulsos STEP gerados para cada eixo.
- p Contagem acumulada de pulsos STEP do atuador.
- p_i Progresso relativo do eixo i em um segmento de movimento.
- PSC Valor do prescaler usado na configuração dos temporizadores.
- $Q16.16$ Formato de ponto fixo com 16 bits para a parte inteira e 16 bits para a parte fracionária.
- $Q16_1$ Representação, no formato $Q16.16$, do valor unitário que corresponde a um passo inteiro acumulado no DDA.
- $\lfloor \cdot \rfloor$ Operador piso, que retorna o maior inteiro menor ou igual ao argumento.
- s_{brake} Distância de frenagem estimada em passos para a rampa trapezoidal.
- $\sigma(e)$ Função de modulação aplicada à velocidade de referência em função do erro de sincronismo.
- $\sigma_{\min}$ Fração mínima admissível da velocidade nominal na modulação cruzada entre eixos.
- θ Posição angular estimada a partir do encoder.
- $e(t)$ Erro instantâneo entre referência e posição medida.
- $u(t)$ Sinal de controle produzido pelo regulador PID.
- $u[k]$ Sinal de controle discreto produzido pelo regulador PID na amostra k .
- K_p, K_i, K_d Ganhos proporcional, integral e derivativo do controlador.
- K Ganho estático do modelo FOPDT.
- K_t Constante de torque do motor.
- L Atraso puro do modelo FOPDT.
- L_f Indutância de fase do motor de passo.
- J Momento de inércia equivalente do eixo controlado.
- R_f Resistência de fase do motor de passo.
- θ_s Posição elétrica comandada pelo campo estático.
- T_L Torque de carga refletido no eixo do motor de passo.

T_e	Torque eletromagnético produzido pelo motor.
T_s	Período de amostragem do laço de controle.
τ	Constante de tempo do modelo FOPDT.
U	Amplitude do degrau aplicado na identificação experimental.
$\bar{v}_{\text{cmd}}, \bar{v}_{\text{enc}}$	Velocidades médias em regime usadas na estimação do ganho estático experimental.
$v_f(t)$	Tensão aplicada à fase do motor.
V_{bus}	Tensão do barramento que alimenta os drivers TMC5160.
ω	Velocidade angular do eixo.

Lista de Abreviaturas

ABN	Canais A, B e N de um encoder incremental.
API	<i>Application Programming Interface.</i>
APB	<i>Advanced Peripheral Bus.</i>
APP	Camada de aplicação do firmware.
ARR	<i>Auto-Reload Register.</i>
BLDC	<i>Brushless Direct Current.</i>
CPS	<i>Counts Per Second.</i>
CNC	Controle Numérico Computadorizado.
CPHA	<i>Clock Phase.</i>
CPOL	<i>Clock Polarity.</i>
CPR	<i>Counts Per Revolution.</i>
CRC	<i>Cyclic Redundancy Check.</i>
CSV	<i>Comma-Separated Values.</i>
DDA	<i>Digital Differential Analyzer.</i>
DMA	<i>Direct Memory Access.</i>
EMC	<i>Electromagnetic Compatibility.</i>
EOF	<i>End of Frame</i> (fim do quadro).
E-STOP	Parada de emergência.
FPGA	<i>Field-Programmable Gate Array.</i>
FOC	<i>Field-Oriented Control.</i>

FOPDT *First-Order Plus Dead Time.*

GUI *Graphical User Interface.*

GPIO *General Purpose Input/Output.*

HAL *Hardware Abstraction Layer.*

IAE *Integral of Absolute Error.*

ISR *Interrupt Service Routine.*

JSON *JavaScript Object Notation.*

LDO *Low-Dropout Regulator.*

LPTIM *Low-Power Timer.*

NVIC *Nested Vectored Interrupt Controller.*

PCB *Printed Circuit Board.*

PID *Proporcional-Integral-Derivativo.*

PMSM *Permanent Magnet Synchronous Motor.*

PSC *Prescaler.*

PWM *Pulse Width Modulation.*

RMS *Root Mean Square.*

RPS *Revolutions Per Second.*

SOF *Start of Frame (início do quadro).*

SoC *System on Chip.*

SPI *Serial Peripheral Interface.*

STEP/DIR/EN *Sinais de passo, direção e habilitação dos drivers de motor de passo.*

SWV *Serial Wire Viewer.*

TIM *Temporizador do microcontrolador STM32.*

TTL *Transistor-Transistor Logic.*

UART *Universal Asynchronous Receiver/Transmitter.*

UFAL Universidade Federal de Alagoas.

USART *Universal Synchronous/Asynchronous Receiver/Transmitter.*

VCP *Virtual COM Port.*

Conteúdo

1	Introdução	1
1.1	Objetivos	1
1.1.1	Objetivo Geral	1
1.1.2	Objetivos Específicos	2
1.2	Organização do texto	2
2	Fundamentação Teórica	3
2.1	Sistemas CNC	3
2.2	Temporizadores do STM32L475	3
2.3	Digital Differential Analyzer	4
2.4	Modelagem de motores de passo	4
2.5	Controlador PID digital	5
2.6	Modelo FOPDT e síntese de ganhos	6
2.7	Integração PID-DDA	6
2.8	Sincronização Cruzada de Eixos	7
2.9	Comunicação SPI e USART	8
2.10	Driver Trinamic TMC5160 e Encoder TMCS-28	11
2.10.1	TMC5160	11
2.10.2	TMCS-28 (Trinamic/Analog Devices)	12
2.10.3	Implicções de projeto	13
2.11	Motores de Passo NEMA 23, Passo de 0,9° e Seleção da Corrente I_{RMS}	13
2.11.1	NEMA 23: geometria e implicações mecânicas	13
2.11.2	Passo elétrico: 1,8° vs 0,9°	13
2.11.3	Microstepping, suavidade e taxa de passos	13
2.11.4	Parâmetros elétricos e modelo de primeira ordem	14
2.11.5	Torques relevantes	14
2.11.6	Resumo para os motores usados	14
2.11.7	Fundamentação teórica para a seleção de I_{RMS}	14
2.12	Registros de Configuração Utilizados	15
2.12.1	STM32L475	15
2.12.2	TMC5160	15

3	Metodologia	18
3.1	Roteiro incremental de bring-up	18
3.2	Arquitetura de software	18
3.2.1	Fluxo do comando <code>cnc-cli</code>	19
3.3	Procedimentos de teste	20
3.4	Identificação experimental do modelo FOPDT	20
3.4.1	Pré-processamento e séries derivadas	20
3.4.2	Extração de marcos temporais	21
3.4.3	Estimativas de K , L e τ	21
3.4.4	Síntese PD e validação	21
3.4.5	Validação gráfica do modelo	21
3.5	Arquitetura de Hardware	22
3.5.1	Visão geral do sistema	23
3.5.2	Alimentação e EMC	23
3.5.3	Drivers TMC5160	24
3.5.4	Encoder óptico TMCS-28	24
3.5.5	Intertravamentos e segurança	25
3.5.6	PCB, cabeamento e montagem	25
3.6	Arquitetura de Controle de Movimento	26
3.6.1	Interface STEP/DIR e tempos do TMC5160	26
3.6.2	MicroPlyer, resolução e DEDGE	27
3.7	DDA em Ponto Fixo (TIM6 @ 50 kHz)	28
3.8	Rampa Trapezoidal (TIM7 @ 1 kHz)	30
3.9	Controle PI de Posição com Encoder	31
3.10	Fila de Movimentos e Segmentação	32
3.11	Mapeamento para o TMC5160	32
3.11.1	Geração de STEP/DIR	32
3.11.2	Resolução de microstepping e MicroPlyer	32
3.12	Parâmetros-chave do Projeto	33
3.13	Coordenação Multi-eixos	33
3.14	Simulador Interativo em Python	34
3.15	Boas práticas e Diagnóstico	35
4	Resultados e Discussão	37
4.1	Desempenho dos serviços principais	37
4.2	Análise do pipeline SPI	37
4.3	Comparação entre simulação e experimento com carga	38
4.4	Integração com a Raspberry Pi	39
5	Conclusão	41

Capítulo 1

Introdução

A popularização de máquinas de Controle Numérico Computadorizado (CNC) depende de controladores capazes de converter instruções digitais em movimentos sincronizados com elevado grau de previsibilidade. No contexto de manufatura de pequeno e médio porte, soluções baseadas em computadores pessoais apresentam custo acessível, mas sofrem com a variabilidade de latência inerente aos sistemas operacionais de propósito geral. Este trabalho investiga uma alternativa embarcada utilizando o microcontrolador STM32L475, capaz de operar a 80 MHz e oferecer temporizadores avançados para geração de pulsos e amostragem de dados [STMicroelectronics, 2013]. Ao combinar a unidade embarcada com uma Raspberry Pi responsável pela interface de alto nível, busca-se garantir escalabilidade e facilidade de integração com pipelines de produção.

A motivação está ligada ao desenvolvimento de um controlador determinista que gere pulsos STEP/DIR/EN em 50 kHz, execute o laço PID a 1 kHz e mantenha comunicação confiável com o host via SPI e USART, entregando registros de telemetria para diagnósticos. A arquitetura proposta precisa coordenar três eixos de motores de passo monitorados por encoders de alta resolução, sincronizar serviços de homing e segurança, e permitir a inserção de novas rotinas sem comprometer as janelas temporais estabelecidas.

O código-fonte, os arquivos de configuração e a documentação de apoio do projeto estão organizados no repositório https://github.com/vjdias/stm32_cnc_controller, de modo que a continuação do trabalho possa partir da mesma base de firmware e testes descrita neste texto.

1.1 Objetivos

1.1.1 Objetivo Geral

Projetar e documentar um controlador CNC determinístico baseado no STM32L475, integrando geração DDA de pulsos, controle PID em tempo real e protocolo de comunicação SPI com um cliente Raspberry Pi.

1.1.2 Objetivos Específicos

- Configurar o temporizador TIM6 para gerar pulsos em 50 kHz utilizando o método DDA.
- Implementar o loop de controle PID a 1 kHz no TIM7, incorporando leitura incremental dos encoders.
- Estruturar o firmware em camadas modulares (Core, App e Services) que facilitem a validação incremental.
- Validar o pipeline de comunicação SPI escravo com DMA circular e logs assíncronos via USART1.
- Registrar testes que comprovem baixa variação temporal (*jitter*), isto é, pequena diferença entre o instante esperado e o instante real de execução dos serviços críticos.

1.2 Organização do texto

O Capítulo 2 apresenta a fundamentação teórica sobre CNC, temporizadores STM32, DDA, modelagem de motores de passo, controle PID, modelo FOPDT e protocolos de comunicação. O Capítulo 3 descreve a metodologia de *bring-up* incremental utilizada para configurar o firmware e o hardware de suporte. Em seguida, o Capítulo 4 reúne os resultados experimentais, analisando desempenho dos serviços e o comportamento do pipeline SPI. Por fim, o Capítulo 5 sintetiza as contribuições, limitações e linhas futuras de trabalho.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta os conceitos fundamentais utilizados na implementação do controlador CNC embarcado. São abordados os princípios de sistemas CNC, a configuração de temporizadores no STM32L475, os métodos DDA para geração de pulsos, a modelagem de motores de passo, os controladores PID, o modelo FOPDT usado na identificação experimental e os aspectos de comunicação SPI/USART empregados no projeto.

2.1 Sistemas CNC

Máquinas de Controle Numérico Computadorizado interpretam instruções codificadas (por exemplo, G-code) e convertem essas ordens em movimentos coordenados entre múltiplos eixos, garantindo precisão repetível no processo de usinagem ou manufatura [Groover, 2015]. Controladores modernos combinam processamento em tempo real com interfaces de alto nível para planejar trajetórias, compensar erros e monitorar a execução. A adoção de arquiteturas híbridas—com uma unidade embarcada responsável pelo tempo real e um computador auxiliar para supervisão—diminui a suscetibilidade a variações temporais e a perdas de sincronismo, mantendo a flexibilidade de integração com sistemas de gestão.

2.2 Temporizadores do STM32L475

O microcontrolador STM32L475 disponibiliza temporizadores de uso geral e avançado capazes de operar na faixa de 80 MHz com alta resolução temporal. A configuração de um temporizador baseia-se na relação

$$f_{\text{TIM}} = \frac{f_{\text{bus}}}{(PSC + 1)(ARR + 1)}, \quad (2.1)$$

onde f_{bus} representa a frequência do barramento APB, PSC o prescaler e ARR o registrador de auto-reload. O guia de aplicação oficial descreve como esses parâmetros possibilitam gerar bases de tempo precisas para laços de controle, captura de entradas e geração de pulsos PWM [STMicroelectronics, 2013]. No projeto, o TIM6 é configurado com $PSC = 79$ e $ARR = 19$ para obter interrupções em 50 kHz, enquanto o TIM7 opera com $PSC = 7999$ e $ARR = 9$, produzindo um laço de 1 kHz. Os temporizadores LPTIM1, TIM3 e TIM5 operam em modo encoder para rastrear incrementalmente a posição dos eixos.

2.3 Digital Differential Analyzer

Algoritmos DDA são integradores digitais que aproximam trajetórias contínuas por meio de incrementos discretos, amplamente utilizados em sistemas de gráficos e em controladores de movimento para motores de passo [Fussell, 2003]. O método acumula um erro fracionário em cada iteração e emite um pulso quando a soma ultrapassa um limiar definido, resultando em uma sequência de passos que aproxima a velocidade ou a trajetória desejada. Arquiteturas clássicas de interpolação tratam o DDA como núcleo do gerador de pulsos, responsável por alimentar os laços de posição e velocidade que seguem a referência calculada amostra a amostra [IDC Technologies, 2014, Koren, 1978, Wang and Hu, 2016, Dronacharya Group of Institutions, 2019]. Panoramas comparativos mostram como variantes circular, linear e por superfície mantêm avanço constante mesmo em trajetórias multi-eixo, o que fundamenta a escolha de incrementos acumulativos sincronizados para o firmware do STM32 [Koren, 2010]. No contexto deste trabalho, o DDA implementado no TIM6 gera sinais STEP com resolução de 20 μs e tolera ajustes de velocidade em tempo real sem quebrar a coesão dos múltiplos eixos. A abordagem permite sincronizar movimentos lineares e circulares através da atualização de incrementos acumulados por eixo durante a ISR do temporizador, preservando a planicidade de avanço descrita pelas referências.

2.4 Modelagem de motores de passo

Motores de passo híbridos apresentam dinâmica eletromecânica dominada por indutâncias de fase, resistência de enrolamento e um torque relacionado à diferença angular entre rotor e campo magnético. Uma forma clássica de representar uma fase do motor é

$$v_f(t) = R_f i_f(t) + L_f \frac{di_f(t)}{dt} + e_f(t), \quad (2.2)$$

em que v_f é a tensão aplicada à fase, i_f é a corrente, R_f e L_f são a resistência e a indutância do enrolamento, e e_f representa a força contraeletromotriz induzida pelo movimento. O

torque resultante depende da corrente nas fases e da diferença entre a posição elétrica comandada e a posição real do rotor, podendo ser resumido por

$$T_e(t) = K_t i_f(t) \sin(\theta_s(t) - \theta(t)). \quad (2.3)$$

A parte mecânica é descrita por

$$J \frac{d\omega(t)}{dt} + B\omega(t) = T_e(t) - T_L(t), \quad (2.4)$$

com J representando a inércia equivalente, B o atrito viscoso, ω a velocidade angular e T_L o torque de carga refletido no eixo [Kenjo and Sugawara, 1994].

Essa modelagem difere da adotada em motores de corrente contínua convencionais porque o motor de passo não é tratado apenas como uma máquina com torque proporcional à corrente de armadura. A posição do campo estatórico também é uma variável de comando, avançando em passos ou micropassos. Por isso, a relação entre torque, posição e corrente é mais acoplada, e efeitos como perda de passo, ressonância e erro de sincronismo passam a ser relevantes para o controle. A utilização de encoders em modo quadratura provê realimentação adicional para compensar perda de passos e acúmulo de erro estático.

2.5 Controlador PID digital

O controlador Proporcional-Integral-Derivativo (PID) continua sendo uma das estratégias mais difundidas para controle de processos, combinando uma ação proporcional que reage ao erro instantâneo, um termo integral que remove erro estacionário e um termo derivativo que prevê tendências de variação [Åström and Hägglund, 1995]. Loops servo em máquinas CNC seguem as posições discretizadas pelo interpolador e corrigem desvios com ganhos sintonizados para cada eixo, podendo incluir observadores ou compensação de distúrbios para manter a precisão em alta velocidade [Wang and Hu, 2016, Koren, 1980, Kung et al., 2005]. Para implementação digital, é comum utilizar a forma incremental

$$u[k] = u[k-1] + K_p(e[k] - e[k-1]) + K_i T_s e[k] + \frac{K_d}{T_s}(e[k] - 2e[k-1] + e[k-2]), \quad (2.5)$$

onde $e[k]$ é o erro discreto entre referência e medição na amostra k , $u[k]$ é o sinal de controle calculado para essa mesma amostra, e T_s é o período de amostragem. Os termos $e[k] - e[k-1]$ e $e[k] - 2e[k-1] + e[k-2]$ correspondem, respectivamente, às diferenças finitas de primeira e segunda ordem usadas para representar as ações proporcional e derivativa na forma incremental. Assim, o último termo não introduz uma segunda derivada física no motor; ele aparece porque a forma incremental calcula a variação do sinal de controle a partir da derivada discreta do erro.

No laço de 1 kHz, o período fixo reduz o esforço computacional e facilita a análise de estabilidade. Estratégias derivadas, como anti-*windup* e filtros de primeira ordem no termo derivativo, são essenciais para lidar com o ruído proveniente dos encoders e das variações do torque de carga. Pesquisas recentes destacam ainda controladores PID com controle cruzado de eixos (*cross-coupled*) que tratam o erro de contorno entre eixos como variável adicional, reduzindo desvios em trajetórias complexas [Wang et al., 2021].

2.6 Modelo FOPDT e síntese de ganhos

Para aproximar a resposta de velocidade em cada eixo, considera-se o modelo de primeira ordem com atraso puro (FOPDT, do inglês *First-Order Plus Dead Time*). Sua função de transferência é escrita como

$$G_v(s) = \frac{Ke^{-Ls}}{\tau s + 1}, \quad (2.6)$$

em que K é o ganho estático, L é o atraso puro e τ é a constante de tempo. Para uma entrada degrau de amplitude U aplicada após o atraso, $u(t) = UH(t - L)$, a resposta no tempo é

$$y(t) = KU (1 - e^{-(t-L)/\tau}) H(t - L). \quad (2.7)$$

A presença de $H(t - L)$ indica explicitamente que a saída permanece nula antes do atraso. Quando $t < L$, tem-se $H(t - L) = 0$ e, portanto, $y(t) = 0$. Quando $t \geq L$, o degrau unitário assume valor 1; nessa condição, o termo $H(t - L)$ deixa de alterar a expressão e a resposta passa a convergir exponencialmente para KU . Essa separação evita a impressão de que a função de Heaviside desaparece da solução: ela apenas passa a valer 1 no trecho posterior ao atraso.

2.7 Integração PID-DDA

A sincronização entre o DDA e o controlador PID ocorre ao transformar o comando de posição desejada em incrementos de passos por período do TIM6. Métodos de distribuição diferencial garantem que ajustes produzidos pelo PID sejam refletidos sem rupturas na geração de pulsos, mantendo o alinhamento entre eixos [Mori et al., 2005, Wang and Hu, 2016]. Materiais didáticos e relatórios industriais ilustram o fluxo completo: o interpolador entrega referências de posição, o encoder fornece o retorno real e o PID calcula o esforço aplicado ao motor para cancelar o erro, em um ciclo repetido a cada 1 ms [IDC Technologies, 2014, Dronacharya Group of Institutions, 2019]. Implementações modernas combinam esses blocos em FPGAs ou SoCs para reduzir latência e habilitar interpolação multi-eixo síncrona com laços servo dedicados [Kung et al., 2005,

Koren, 1980]. No firmware do STM32, o laço de controle alimenta as metas de velocidade e microstepping de cada eixo, enquanto o DDA executa as transições discretas, possibilitando perfis suaves e respeitando os limites de aceleração definidos pelo firmware.

2.8 Sincronização Cruzada de Eixos

Em movimentos multi-eixo, a coordenação cinemática exige que cada eixo siga uma fração coerente do avanço global, minimizando erro de sincronismo e erro de contorno. Neste trabalho, o termo *sincronização cruzada de eixos* designa o conjunto de estratégias que, sobre as malhas de posição e velocidade por eixo, impõem coerência temporal ao grupo, em linha com abordagens *cross-coupled* descritas na literatura.

Sejam $s_i(t)$ as posições desejadas e $x_i(t)$ as posições medidas. Um progresso normalizado por eixo pode ser escrito como

$$p_i(t) = \frac{x_i(t) - x_i(0)}{\max\{1, |s_i(T) - s_i(0)|\}}, \quad (2.8)$$

em que T representa a duração nominal do movimento. Uma escolha natural para o eixo mestre é aquele com maior restante absoluto, ou de forma equivalente, menor progresso relativo. O erro de sincronismo em um eixo não mestre pode ser expresso por

$$e_{\text{sync},i}(t) = p_m(t) - p_i(t), \quad i \neq m. \quad (2.9)$$

Uma lei simples de modulação de avanço reescala a velocidade de referência dos eixos não mestres:

$$v_{\text{ref},i}(t) = \sigma(e_{\text{sync},i}(t)) v_{\text{nom},i}(t), \quad (2.10)$$

com

$$\sigma(e) = 1 - (1 - \sigma_{\min}) \min \left\{ 1, \frac{|e|}{e_{\text{th}}} \right\}, \quad (2.11)$$

em que $\sigma_{\min} \in (0, 1)$ define a fração mínima admissível de velocidade e e_{th} é um limiar de sincronismo. Em situações críticas, uma retenção de sincronismo pode impor $v_{\text{ref},i} = 0$ até que a defasagem volte a uma faixa segura.

Durante a fase final do movimento, convém adotar uma janela de acabamento mais conservadora, com menor troca de mestre e maior tolerância a pequenas variações. Essa sincronização cruzada não substitui as malhas PID por eixo; ela atua como um supervisor de alto nível que gera referências coerentes e respeita limitações de aceleração e saturação.

2.9 Comunicação SPI e USART

A comunicação com a Raspberry Pi utiliza o periférico SPI2 em modo escravo com DMA circular. Esse desenho reduz a carga da CPU e garante que as transferências de 42 bytes sejam executadas dentro da janela entre interrupções do TIM6 e TIM7. A USART1, configurada como *Virtual COM Port*, é utilizada para depuração e registro de eventos críticos, com uma fila não bloqueante que impede o impacto sobre o laço de controle. A coordenação entre SPI e USART é fundamental para evitar inversões de prioridade que poderiam comprometer o determinismo do sistema [STMicroelectronics, 2018b]. A análise do pipeline de polls evidencia como o firmware pausa e reinicia o DMA para garantir que cada resposta seja transmitida somente após processamento completo.

A escolha do SPI para comandos foi motivada pela disponibilidade do barramento na Raspberry Pi, pelo tamanho curto dos quadros e pela possibilidade de usar DMA no STM32. A USART permaneceu separada do canal de comando para servir como caminho de observabilidade: logs podem ser perdidos ou reduzidos em cenários de carga, mas não devem bloquear a execução do movimento.

SPI Raspberry Pi ↔ STM32: protocolo do enlace

O enlace SPI escravo (STM32 como escravo) utiliza quadros de tamanho fixo com **42 bytes**, transferidos em modo *full-duplex*. O mestre (Raspberry Pi) envia um *poll* com a requisição e recebe, no mesmo ciclo ou nos ciclos subsequentes, a resposta preparada pelo firmware (via fila de respostas e DMA). O quadro adota marcadores de início/fim para robustez:

Tabela 2.1: Layout do quadro SPI (tamanho fixo: 42 bytes).

Offset	Tamanho	Campo	Descrição
0	1	SOF	Byte fixo de início (0xAB) para sincronização do quadro.
1	1	CMD	Identificador do serviço/comando (ex.: 0x10=LED, 0x20=MOVE).
2	1	FLAGS	Bits de controle/opções do comando (reservado = 0x00).
3	1	LEN	Comprimento válido do PAYLOAD (0 a 34 bytes).
4	34	PAYLOAD	Dados da requisição; preencher com 0x00 quando não usado.
38	2	CRC16	Verificação de integridade (LSB,MSB) calculada sobre bytes 0..37.
40	1	RSV	Reservado (0x00) para alinhamento/expansão futura.
41	1	EOF	Marcador de fim: 0x54.

Quando não há resposta pronta, o escravo devolve um quadro com todos os 42 bytes preenchidos com 0xA5. Quando a resposta de um serviço está completa, o escravo publica o quadro final com EOF=0x54 e CRC válido, conforme observado no cliente `cnc_spi_client.py`.

Serviço LED (exemplo de comando simples)

O serviço LED liga/desliga ou alterna o estado. O mestre envia uma requisição; a resposta confirma o estado final.

Tabela 2.2: Requisição LED (CMD=0x10).

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x10 (LED)
2	1	FLAGS	0x00 (padrão)
3	1	LEN	0x02
4	1	LED_ID	0x00 (LED on-board)
5	1	MODE	0x00=OFF, 0x01=ON, 0x02=TOGGLE
6..37	32	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Tabela 2.3: Resposta LED.

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x10 (eco)
2	1	FLAGS	0x00
3	1	LEN	0x02
4	1	STATUS	0x00=OK, >0=erro
5	1	LED_STATE	0x00=OFF, 0x01=ON
6..37	32	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Serviço Movimento (exemplo de comando composto)

O serviço de movimento recebe metas por eixo e parâmetros cinemáticos simplificados. Neste trabalho, a cinemática usada pelo firmware é cartesiana direta: os campos X_STEPS, Y_STEPS e Z_STEPS já chegam como deslocamentos relativos por eixo, enquanto FEED limita a velocidade máxima do segmento. Assim, a conversão geométrica mais complexa fica fora deste quadro e o serviço de movimento se concentra em enfileirar passos, sentido e limite de velocidade. Campos não usados devem ser zero.

Tabela 2.4: Requisição Movimento (CMD=0x20).

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x20 (MOVE)
2	1	FLAGS	0x00
3	1	LEN	Deve refletir o total de bytes úteis do payload (exemplo: 0x15).
4	1	AXIS_MASK	Máscara de eixos: Bit0=X, Bit1=Y, Bit2=Z. Permite ativar eixos individualmente.
5..8	4	X_STEPS	int32 (passos relativos) Positivo/negativo define o sentido.
9..12	4	Y_STEPS	int32 (passos relativos) Positivo/negativo define o sentido.
13..16	4	Z_STEPS	int32 (passos relativos) Positivo/negativo define o sentido.
17..20	4	FEED	Velocidade máxima alvo em passos/s (uint32). Limitador para o perfil de movimento.
21	1	JERK	Parâmetro opcional do perfil (suavização). Se não utilizado, enviar 0x00.
22..37	16	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Tabela 2.5: Resposta Movimento.

Offset	Tamanho	Campo	Valor/Descrição
0	1	SOF	0xAB
1	1	CMD	0x20 (eco)
2	1	FLAGS	0x00
3	1	LEN	0x03
4	1	STATUS	0x00=aceito, 0x01=ocupado, >0=erro
5	1	QUEUE_DEPTH	Itens pendentes na fila de movimentos
6	1	RESERVED	0x00
7..37	31	RSV	0x00
38	2	CRC16	CRC-16 sobre bytes 0..37
40	1	RSV	0x00
41	1	EOF	0x54

Notas sobre os campos da resposta MOVE:

- **STATUS:** 0x00=aceito (comando enfileirado/executando); 0x01=ocupado (tentar novamente); valores >0 indicam erro (código específico do firmware).
- **QUEUE_DEPTH:** tamanho atual da fila de movimentos, útil para controle de fluxo do mestre.
- **CRC16:** valida a integridade do quadro; CRC inválido deve ser tratado como erro de comunicação.

SPI Raspberry Pi ↔ TMC5160: configuração por registradores

O TMC5160 expõe um barramento SPI para configuração e diagnóstico por meio de transações de **40 bits** (5 bytes por quadro), compostas por um byte de endereço e quatro bytes de dados. O bit mais significativo do byte de endereço indica leitura/escrita (**1**=leitura, **0**=escrita) e os 7 bits menos significativos selecionam o registrador. O protocolo apresenta **latência de uma transação**: os dados retornados em *SD0* referem-se ao comando emitido no quadro anterior. Recomenda-se emitir uma leitura “de preparo” antes de coletar o valor válido do registrador desejado (ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), sd]). Em implementações usuais o SPI opera em modo 3 (CPOL=1, CPHA=1).

Tabela 2.6: Quadro de escrita TMC5160 (5 bytes, big-endian).

Offset	Tamanho	Campo	Descrição
0	1	ADDR[6:0], R/W=0	Endereço do registrador (7 bits), bit 7=0 (escrita).
1..4	4	DATA[31:0]	Palavra de 32 bits, ordem de bytes: MSB → LSB.

Tabela 2.7: Quadro de leitura TMC5160 (5 bytes, big-endian; resposta no ciclo seguinte).

Offset	Tamanho	Campo	Descrição
0	1	ADDR[6:0], R/W=1	Endereço do registrador (7 bits), bit 7=1 (leitura).
1..4	4	DUMMY	0x00; os 32 bits lidos pertencem ao quadro anterior.

Exemplos de registros comuns na bring-up e operação são apresentados junto ao TMC5160 (Seção 2.10.1).

Os exemplos acima alinham-se ao fluxo de *poll* descrito na Seção 2.9.

2.10 Driver Trinamic TMC5160 e Encoder TMCS-28

Os drivers de motor de passo da Trinamic são amplamente utilizados por oferecerem modos de comutação silenciosos, detecção de carga *sensorless* e parametrização fina via registradores. Em complemento, encoders ópticos incrementais como o TMCS-28 permitem medição sem contato da posição/ângulo do eixo com disco óptico e sensor, habilitando calibrações de zero, telemetria e malhas fechadas. Esta seção resume os recursos relevantes do TMC5160 e do TMCS-28 e como eles se relacionam à arquitetura do firmware.

2.10.1 TMC5160

O TMC5160 é um driver de alto desempenho voltado a correntes elevadas, com interface SPI e controle por pinos STEP/DIR/EN. Entre os principais recursos:

- **Microstepping e microPlyer**: geração de até 256 micropassos e interpolação *microPlyer* a partir de entradas com baixa resolução, suavizando o movimento mesmo com taxas de pulso moderadas.
- **stealthChop2**: modo de chaveamento focado em baixo ruído acústico; configurado principalmente em PWMCONF.
- **spreadCycle**: chopper clássico de corrente para maior fidelidade em torque em altas velocidades; parametrização em CHOPCONF.

- **StallGuard2:** medição de carga sem sensor (*sensorless*) que permite detectar perda de passo/contato; limiar em `SGTHRS` e leitura do ganho em `SG_RESULT`.
- **coolStep:** regulação dinâmica de corrente baseada na carga para reduzir perdas sem comprometer torque.
- **dcStep:** avanço dependente de carga para evitar perda de passo em condições adversas.
- **Proteções e diagnóstico:** `DRV_STATUS` expõe flags de sobrecorrente, subtensão e temperatura.

Na integração com o firmware, o TMC5160 é inicializado via SPI ajustando `IHOLD_IRUN` (correntes de hold/run), `TPOWERDOWN`, `CHOPCONF` e `PWMCONF`. A comutação de perfis (*stealthChop2* $\leftrightarrow$ *spreadCycle*) pode ser feita em tempo de execução para equilibrar ruído e robustez. Para homing *sensorless*, calibra-se `SGTHRS` e a janela `TCOOLTHRS` de maneira a obter detecção repetível sem falsos positivos.

Tabela 2.8: Exemplos de acessos SPI ao TMC5160.

Registro	Operação/Observação
<code>IHOLD_IRUN</code>	Escrita dos campos <code>IHOLD</code> , <code>IRUN</code> e <code>IHOLDDELAY</code> para corrente de hold/run e rampa.
<code>CHOPCONF</code>	Seleção de <i>spreadCycle</i> ou parâmetros de histerese do chopper.
<code>PWMCONF</code>	Parâmetros do <i>stealthChop2</i> (<code>PWM_AMPL</code> , <code>PWM_GRAD</code>).
<code>SGTHRS</code>	Limiar do <i>StallGuard2</i> para homing sensorless.
<code>DRV_STATUS</code>	Leitura de flags de falha e <code>SG_RESULT</code> ; lembrar da latência de um quadro.

2.10.2 TMCS-28 (Trinamic/Analog Devices)

O TMCS-28 é um encoder óptico incremental de baixo custo e dimensão reduzida para motores de passo e PMSM/BLDC. Utiliza roda código óptica e fornece saídas em quadratura **A** e **B** mais **N** (*index*). Principais pontos ([TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), 2022]):

- **Resoluções:** até 625 lpr (*lines per rotation*) $\Rightarrow$ 40 000 cpr; opção de 64 lpr $\Rightarrow$ 4 096 cpr.
- **Sinal:** ABN em nível TTL, *rise/fall* típicos 10 ns, $V_{OH} \geq 2.4$ V, $V_{IL} \leq 0.4$ V, $I_{out\ max} \approx 20$ mA.
- **Alimentação:** 4.5–5.5 V (típico 5 V), consumo ≈ 110 mA.
- **Faixa de frequência:** até ~ 1.5 MHz de comutação.
- **Mecânica:** diâmetro do furo 5 mm ou 6.35 mm; carga axial 50 N, radial 80 N; rotação máxima 6000 rpm; módulo 28 mm $\times$ 28 mm $\times$ 18 mm (aprox.).

Integração com o firmware: conectar A/B aos temporizadores do STM32 em modo *encoder* (por exemplo, LPTIM1/TIM3/TIM5) e o *index* N a uma entrada de reset/captura para referência de zero. Converter contagens em ângulo via $\theta = 360^\circ \cdot \text{count}/\text{CPR}$. Neste TCC foi utilizada a variante **TMCS-28-10k** (código TMCS-28-x-10000-AT-01), com **625 lpr** e **40 000 cpr**; portanto, considerar $\text{CPR} = 40\,000$. Como as saídas são TTL a 5 V, prever adaptação de nível para GPIOs de 3.3 V do STM32 (divisores/*level shifter*).

2.10.3 Implicções de projeto

No contexto deste trabalho, os recursos de microstepping e os modos *stealthChop2/spreadCycle* são os mais relevantes para compatibilizar a taxa de passos do DDA (50 kHz) com suavidade e torque. Quando aplicável, o uso de *StallGuard2* permite procedimentos de homing sem sensores, desde que a calibração de *SGTHRS/TCOOLTHRS* seja validada em bancada. A parametrização via SPI/UART se integra naturalmente à camada de *Services*, mantendo os ajustes desacoplados do laço de tempo real.

2.11 Motores de Passo NEMA 23, Passo de 0,9° e Seleção da Corrente I_{RMS}

2.11.1 NEMA 23: geometria e implicações mecânicas

NEMA 23 define as dimensões da *flange* do motor (aproximadamente 57 mm × 57 mm) e o padrão de furação de montagem, mas não define torque, corrente nominal ou passo elétrico. Assim, motores com essa carcaça podem variar em comprimento do corpo, inércia do rotor, resistência e indutância por fase, além do torque de retenção. No projeto deste trabalho, os três eixos utilizam motores NEMA 23 por combinarem: (i) facilidade de fixação em estruturas CNC; (ii) bom compromisso entre torque e tamanho; e (iii) ampla disponibilidade de *drivers* compatíveis, como o TMC5160.

2.11.2 Passo elétrico: 1,8° vs 0,9°

O ângulo de passo elétrico corresponde à rotação do eixo a cada passo inteiro sem *microstepping*. Os valores mais usuais são 1,8° e 0,9°, o que leva a

$$N_{\text{passos/rev}} = \frac{360^\circ}{\theta_{\text{passo}}}. \quad (2.12)$$

Assim, motores de 1,8° possuem 200 passos por volta, enquanto os de 0,9° possuem 400 passos por volta.

Na prática, o passo de 0,9° oferece maior resolução mecânica por volta e reduz o *ripple* posicional em baixas velocidades. Em contrapartida, exige taxa de pulsos mais alta para a mesma velocidade angular. Para um fator de microstepping M , vale

$$\text{RPS} = \frac{f_{\text{STEP}}}{N \cdot M} \Rightarrow f_{\text{STEP}} = \text{RPS} \cdot N \cdot M. \quad (2.13)$$

Portanto, a mesma rotação por segundo em um motor de 0,9° demanda o dobro da frequência de pulsos em relação a um motor de 1,8°.

2.11.3 Microstepping, suavidade e taxa de passos

O *microstepping* interpola correntes aproximadamente senoidais nas fases do motor, reduzindo ruído, vibração e rugosidade de movimento. Entretanto, a resolução efetiva de comando continua limitada pela frequência de pulsos *STEP* disponível no controlador. Além disso, o torque incremental associado a cada micropasso é menor do que o torque de passo inteiro, razão pela qual a operação com cargas dinâmicas exige correntes compatíveis com a aceleração desejada e com a curva torque versus velocidade do motor.

2.11.4 Parâmetros elétricos e modelo de primeira ordem

Motores de passo bipolares são especificados por corrente por fase I_{rated} , resistência R e indutância L . Em primeira aproximação, a dinâmica de corrente da fase pode ser escrita como

$$\frac{di}{dt} \approx \frac{V_{bus} - v_{chopper}}{L}, \quad \tau = \frac{L}{R}. \quad (2.14)$$

No TMC5160, o controle por corrente aplica *chopping* para manter o valor de I_{RMS} desejado, usando tensão de barramento elevada para acelerar a resposta de corrente contra a indutância do enrolamento. O aquecimento cresce aproximadamente com $P \propto I_{RMS}^2 R$, de modo que trabalhar com frações moderadas da corrente nominal reduz a dissipação sem necessariamente comprometer o torque requerido.

2.11.5 Torques relevantes

- **Torque de retenção** (T_{hold}): torque estático máximo com corrente nominal e eixo travado.
- **Torque de detente** (T_{det}): ondulação mecânica intrínseca do rotor sem corrente aplicada.
- **Torque dinâmico**: decresce com a velocidade elétrica devido à limitação de di/dt e à força contraeletromotriz; por isso, barramentos mais altos ajudam a preservar torque em rotações elevadas.

2.11.6 Resumo para os motores usados

- **JK57HM76-2804**: NEMA 23, corrente nominal de aproximadamente 2.8 A por fase e torque de retenção em torno de 1.8 N m. É adequado aos eixos que exigem maior força estática.
- **23HM8430 0,9°**: NEMA 23, corrente nominal de aproximadamente 3.0 A e torque de retenção próximo de 1.5 N m. O passo fino nativo favorece resolução e suavidade em baixas velocidades.

2.11.7 Fundamentação teórica para a seleção de I_{RMS}

Como primeira aproximação, o torque de retenção pode ser usado para estimar a constante de torque do motor:

$$k_T \approx \frac{T_{hold}}{I_{rated}} \quad [\text{N m A}^{-1}]. \quad (2.15)$$

Com isso, uma estimativa da corrente mínima prática para superar o torque de detente pode ser escrita como

$$I_{min} \approx \alpha \frac{T_{det}}{k_T}, \quad (2.16)$$

em que α é um fator de segurança escolhido entre 1,5 e 2,5 para compensar atrito e irregularidades dinâmicas.

Assumindo $T_{det} \approx 0.06 \text{ N m}$ como valor típico para motores NEMA 23, obtém-se a Tabela 2.9.

Tabela 2.9: Estimativa da corrente mínima prática (I_{RMS}).

Motor	I_{rated} [A]	T_{hold} [N·m]	k_T [N·m/A]	I_{min} [A]
JK57HM76-2804	2,8	1,8	0,643	0,14–0,23
23HM8430 0,9°	3,0	1,5	0,50	0,18–0,30

Do ponto de vista térmico e de robustez, adotou-se no firmware um perfil de corrente reduzida para os testes de bancada, com $I_{RUN} \approx 0.28 \text{ A}$ e $I_{HOLD} \approx 0.12 \text{ A}$ no motor JK57HM76-2804, e $I_{RUN} \approx$

0.30 A com $I_{\text{HOLD}} \approx 0.12 \text{ A}$ a 0.15 A no 23HM8430. Esses valores representam apenas uma fração da corrente nominal, reduzindo expressivamente a potência dissipada sem prejuízo significativo sobre o torque necessário aos movimentos previstos neste protótipo CNC. Caso o processo venha a exigir maior aceleração ou maior esforço mecânico, o ajuste de I_{RUN} pode ser feito dinamicamente no TMC5160 por meio do registrador `IHOLD_IRUN`.

2.12 Registros de Configuração Utilizados

Esta seção sumariza os principais registros configurados no STM32L475 e no driver TMC5160 conforme empregados neste trabalho. O formato segue o exemplo de tabelas de configuração com campos e finalidade.

2.12.1 STM32L475

2.12.2 TMC5160

Observação: valores exatos de bits podem variar conforme a placa/variante e roteiro de testes; recomenda-se validar com os manuais do STM32 L4 ([STMicroelectronics, 2018a]) e o datasheet do TMC5160 ([TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), sd]).

Tabela 2.10: Registros STM32 L4 utilizados neste trabalho (ver [STMicroelectronics, 2013, STMicroelectronics, 2018a]).

Perif.	Registro	Campo/Valor	Finalidade
TIM6	PSC	79	Prescaler para 50 kHz (DDA).
TIM6	ARR	19	Período de 20 μ s.
TIM7	PSC	7999	Prescaler para 1 kHz (PID).
TIM7	ARR	9	Período de 1 ms.
LPTIM1	CFGR	ENC=1; CKPOL=00	Modo encoder externo (A/B) para eixo X.
LPTIM1	ARR	0xFFFF	Período máximo de 16 bits (contagem modular).
TIM3/5	SMCR	SMS=0b011	Modo encoder (contagem em TI1 e TI2).
TIM3/5	CCMR1	CC1S=01; CC2S=01	Entradas mapeadas para TI1/TI2.
TIM3/5	CCER	CC1P=0; CC2P=0	Polaridade não invertida (conforme ligação).
SPI2	CR1	MSTR=0; CPOL=0; CPHA=0	Modo escravo, fase/polaridade padrão.
SPI2	CR2	RXDMAEN=1; TXDMAEN=1; DS=8	SPI com DMA e palavra de 8 bits.
DMA1 (Ch4/Ch5)	CCR(RX/TX)	MINC=1; CIRC=1; DIR	DMA circular para SPI2 RX/TX.
DMAx	CNDTR/CPAR/CMAR	–	Tamanho e endereços de perif. e memória.
USART1	BRR	115200	Baud rate para logs/telemetria.
EXTI	IMR/RTSR/FTSR	Linhas de E-STOP/limites	Interrupções de segurança.
NVIC	PRIOR	–	Priorização: segurança, TIM6, SPI/DMA, TIM7, USART.

Tabela 2.11: Registros TMC5160 utilizados neste trabalho (ver [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), sd]).

Registro	Campo/Valor	Finalidade
IHOLD_IRUN	IHOLD, IRUN, IHOLDDELAY	Correntes de hold/run e rampa de corrente.
TPOWERDOWN	Tempo (μ s)	Tempo para redução de corrente em inatividade.
CHOPCONF	spreadCycle/ hysteresis	Chopper de corrente e modo de comutação.
PWMCONF	stealthChop2 (PWM_AMPL/GRAD)	Chaveamento silencioso e parâmetros PWM.
SGTHRS	Limiar	Limiar do StallGuard2 para homing <i>sensorless</i> .
TCOOLTHRS	Velocidade limiar	Janela de atuação para StallGuard/coolStep.
TPWMTHRS	Velocidade limiar	Limite de comutação stealth-Chop2 $\leftrightarrow$ spreadCycle.
DRV_STATUS	Flags	Diagnóstico: sobrecorrente, subtensão, temperatura, SG_RESULT.

Capítulo 3

Metodologia

A metodologia adotada para o desenvolvimento do controlador CNC seguiu um roteiro incremental que prioriza a validação dos blocos críticos antes de incorporar funcionalidades auxiliares. Cada etapa foi documentada, testada e revisada de forma a garantir que os requisitos de determinismo fossem mantidos durante todo o processo.

3.1 Roteiro incremental de bring-up

O ponto de partida consistiu na configuração do clock principal para 80 MHz e na definição das prioridades do NVIC de acordo com o orçamento temporal: interrupções externas de segurança, TIM6, SPI2/DMA, TIM7 e USART1. Em seguida, foram ativadas as entradas de parada de emergência (E-STOP) e sensores de proximidade, assegurando que flags de segurança pudessem interromper o fluxo de comandos. As etapas posteriores focaram na calibração dos temporizadores: o TIM6 foi dimensionado com $PSC = 79$ e $ARR = 19$, enquanto o TIM7 recebeu $PSC = 7999$ e $ARR = 9$. Os temporizadores LPTIM1, TIM3 e TIM5 foram configurados em modo quadratura para leitura de encoders.

A configuração do SPI2 em modo escravo com DMA circular foi realizada após os temporizadores, evitando contenda na memória compartilhada. Por fim, a USART1 foi ajustada para 115 200 bps e integrada a um serviço de log com fila não bloqueante. Cada etapa do roteiro foi acompanhada por testes de bancada: medições de frequência com osciloscópio, leitura de contadores de encoder e injeção de quadros SPI utilizando o cliente Python.

3.2 Arquitetura de software

O firmware foi estruturado em três camadas principais. A camada *Core* engloba os artefatos gerados pelo STM32CubeMX, incluindo inicialização de periféricos, descrições de pinos e funções HAL. Sobre ela, a camada *App* implementa o laço principal (`app_poll`), o agendador de serviços e as rotinas de inicialização específicas do projeto. A camada *Services* agrupa módulos especializados, como o gerador de passos, o controlador PID, o serviço de homing e o roteador de mensagens SPI.

A fila de recepção SPI é mantida em memória circular, preenchida pelas rotinas de interrupção e consumida por `app_poll`. Respostas são enfileiradas em `g_app_responses` e promovidas para o buffer de DMA quando disponíveis. A integração com os drivers TMC5160 ocorre por meio de uma API dedicada que abstrai comandos STEP/DIR/EN e monitora condições de falha.

A Figura 3.1 resume a arquitetura em blocos, mostrando a separação entre supervisão, comunicação, serviços de aplicação e laços temporizados de controle.

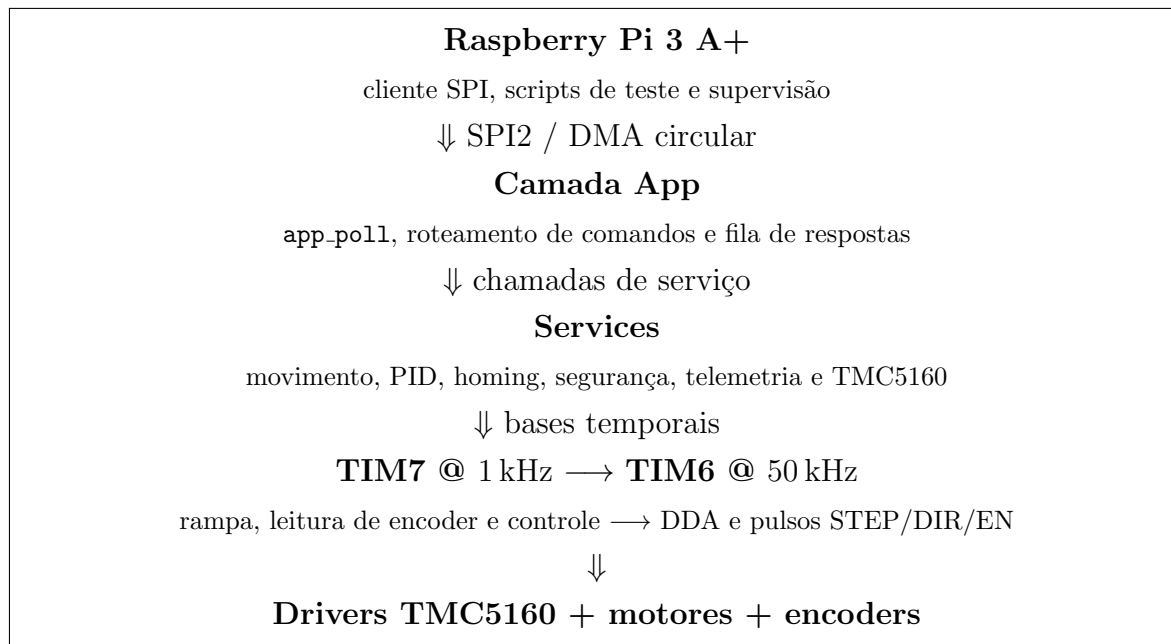


Figura 3.1: Diagrama em blocos da arquitetura de software e controle de movimento.

3.2.1 Fluxo do comando `cnc-cli`

O comando `cnc-cli` executa uma sequência declarativa de ações descritas em um arquivo JSON, incluindo *patches* no TMC5160, ajustes de PID e movimentos. A Figura 3.2 resume o fluxo principal, desde a validação da sequência no host até o encerramento da execução no STM32. Em alto nível, ele envolve:

1. leitura de `application.cfg` e do arquivo de sequência;
2. validação dos limites de PID, velocidade e parâmetros do TMC5160;
3. abertura do enlace SPI com o STM32;
4. sinalização de execução pelo LED do firmware;
5. iteração sobre os passos da sequência (`tmc{}`, `pid{}` e `move{}`);
6. monitoramento periódico do estado da fila de movimentos e dos registradores de diagnóstico do TMC5160;
7. aplicação de rotina de segurança em caso de falha;
8. encerramento normal com desligamento seguro dos estágios de potência.

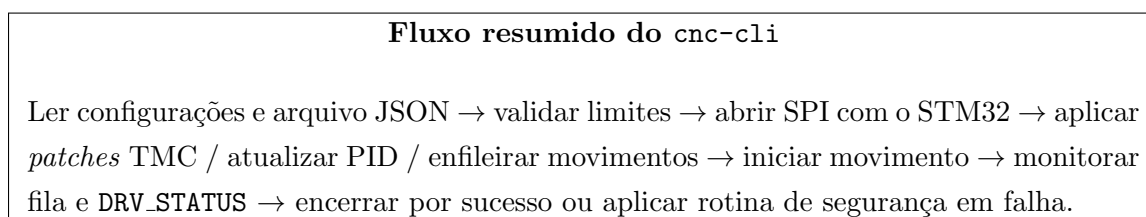


Figura 3.2: Fluxo resumido do comando `cnc-cli`, desde a validação da sequência até o monitoramento e encerramento da execução.

Em termos operacionais:

1. o cliente carrega o `application.cfg` e o JSON da sequência;
2. cada item é verificado contra os limites configurados no host;
3. a comunicação com o STM32 ocorre pelo protocolo CNC sobre SPI;
4. comandos `tmc{...}` aplicam *patches* em registradores como `CHOPCONF`, `PWMCONF` e `IHOLD_IRUN`;
5. comandos `pid{...}` atualizam ganhos padrão por eixo;
6. comandos `move{...}` enfileiram segmentos e disparam o início do movimento;
7. o cliente acompanha o progresso por meio de `MOVE_QUEUE_STATUS` e dos registradores de falha do TMC5160;
8. em caso de sobretemperatura, subtensão ou *driver error*, aplica-se um estado seguro com corrente nula e desligamento dos *drivers*.

No caso específico de `MOVE_QUEUE_STATUS`, a identificação do quadro `frameId` é preservada no terceiro byte útil do protocolo em ambos os sentidos: a requisição tem 4 bytes e usa `raw[2]` como `frameId`, enquanto a resposta tem 12 bytes e também ecoa `frameId` em `raw[2]`. Esse espelhamento facilita correlacionar a consulta enviada pelo host com o estado devolvido pelo firmware.

3.3 Procedimentos de teste

Os testes de validação foram conduzidos em três frentes. Primeiro, medições com osciloscópio e analisador lógico verificaram a frequência dos pulsos STEP e a variação temporal (*jitter*) do TIM6, confirmando a estabilidade em 50 kHz. Em seguida, foram realizadas varreduras de ganho nos controladores PID para avaliar margem de fase e resposta a degraus, utilizando logs exportados via USART1. Por fim, o pipeline SPI foi monitorado com o cliente `cnc_spi_client.py`, observando a necessidade de até três ciclos de enquete para respostas completas e avaliando o impacto de diferentes valores de `APP_SPI_RESTART_DEFER_MAX`. Os dados coletados subsidiam as análises apresentadas no Capítulo 4.

3.4 Identificação experimental do modelo FOPDT

Esta seção descreve o procedimento adotado para estimar os parâmetros K , L e τ da malha de velocidade a partir de respostas a degrau obtidas em bancada. Os ensaios foram conduzidos com o firmware instrumentado para emitir amostras de posição e velocidade via USART e SWV, em formato CSV, a partir das rotinas de telemetria do serviço de movimento. Para cada combinação de eixo e microstepping, foi aplicado um perfil de degrau de velocidade gerado pelo DDA, e os sinais de referência e medição foram registrados para posterior processamento em Python.

O método opera inteiramente no domínio do tempo, em linha com as abordagens clássicas de identificação por curva de reação, e mostrou-se robusto a níveis moderados de ruído.

3.4.1 Pré-processamento e séries derivadas

Considere amostras ordenadas por tempo contendo os campos (t, p, c) , em que p representa a contagem acumulada de pulsos do atuador e c a contagem acumulada do encoder. A partir de pares de amostras, ou de uma janela deslizante de comprimento N , definem-se

$$\Delta t_k = t_k - t_{k-N+1}, \quad \Delta p_k = p_k - p_{k-N+1}, \quad \Delta c_k = c_k - c_{k-N+1}, \quad (3.1)$$

e as velocidades discretas

$$v_{\text{cmd}}(k) = \frac{\Delta p_k}{\Delta t_k}, \quad v_{\text{enc}}(k) = \frac{\Delta c_k}{\Delta t_k}. \quad (3.2)$$

Para reduzir o efeito de ruído, utiliza-se tipicamente $N \in [5, 10]$. Pares com $\Delta t_k \leq 0$ são descartados. O valor de regime de cada série é obtido pela média dos últimos 20% da janela temporal disponível. Em outras palavras, a estimação usa a parte final do ensaio como aproximação do patamar em regime, o que torna a extração de ganho menos sensível a flutuações instantâneas.

3.4.2 Extração de marcos temporais

Define-se o início efetivo do degrau nos sinais de comando e de medição pelos primeiros cruzamentos de 5% dos respectivos regimes:

$$t_{5\%}^{\text{cmd}} = \inf \{t : v_{\text{cmd}}(t) \geq 0,05\bar{v}_{\text{cmd}}\}, \quad t_{5\%}^{\text{enc}} = \inf \{t : v_{\text{enc}}(t) \geq 0,05\bar{v}_{\text{enc}}\}. \quad (3.3)$$

De forma análoga, o instante t_{63} é o primeiro cruzamento de 63,2% do regime medido:

$$t_{63} = \inf \{t : v_{\text{enc}}(t) \geq 0,632\bar{v}_{\text{enc}}\}. \quad (3.4)$$

3.4.3 Estimativas de K , L e τ

O ganho estático do modelo é calculado pela razão entre os regimes do encoder e do comando:

$$K = \frac{\bar{v}_{\text{enc}}}{\bar{v}_{\text{cmd}}}. \quad (3.5)$$

O tempo morto é estimado por

$$L = \max \{0, t_{5\%}^{\text{enc}} - t_{5\%}^{\text{cmd}}\}, \quad (3.6)$$

enquanto a constante de tempo decorre de

$$\tau = t_{63} - L. \quad (3.7)$$

Quando limitações de amostragem impõem baixa resolução temporal, adota-se como piso o menor intervalo temporal observado na série crua. Isso evita estimativas nulas ou negativas de L e τ , que seriam incompatíveis com o modelo FOPDT adotado.

3.4.4 Síntese PD e validação

Com os parâmetros (K, L, τ) , computam-se os ganhos K_p e K_d pelas regras de Ziegler–Nichols para curva de reação, mantendo $K_i = 0$ na malha de velocidade. Em seguida, os ganhos contínuos são escalonados para o formato inteiro utilizado no firmware, preservando a mesma convenção adotada no controlador discreto executado no STM32.

A validação consiste em aplicar as constantes ao modelo FOPDT discretizado com $T_s = 1$ ms e comparar a resposta simulada com a resposta medida, observando sobre-elevação, tempo de subida e ausência de saturações relevantes.

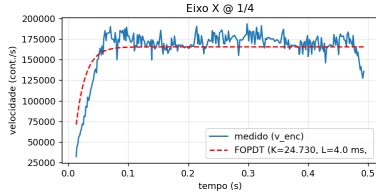
3.4.5 Validação gráfica do modelo

Para documentar a aderência do modelo, foram gerados gráficos de sobreposição entre a velocidade medida pelo encoder e a resposta teórica do FOPDT a um degrau. A resposta empregada decorre da

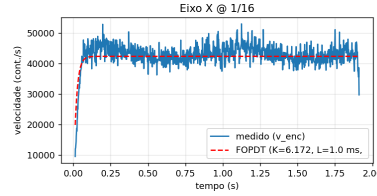
solução analítica do sistema de primeira ordem com atraso:

$$y(t) = \begin{cases} 0, & t < L, \\ KU (1 - e^{-(t-L)/\tau}), & t \geq L, \end{cases} \quad (3.8)$$

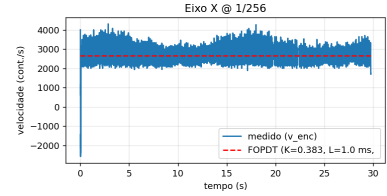
em que a amplitude U é tomada como a velocidade de comando em regime. As Figuras 3.3, 3.4 e 3.5 mostram a comparação para os eixos X, Y e Z nos três níveis de microstepping avaliados.



(a) Eixo X @ 1/4.

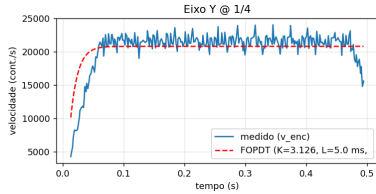


(b) Eixo X @ 1/16.

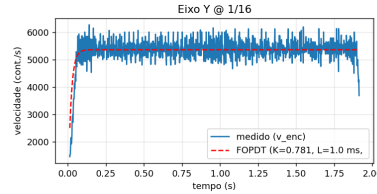


(c) Eixo X @ 1/256.

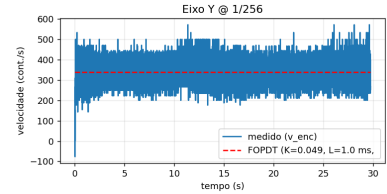
Figura 3.3: Sobreposição entre velocidade medida e resposta FOPDT para o eixo X.



(a) Eixo Y @ 1/4.

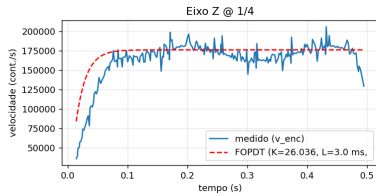


(b) Eixo Y @ 1/16.

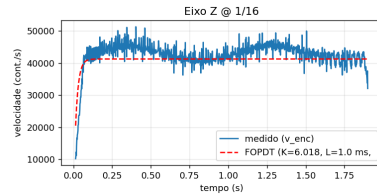


(c) Eixo Y @ 1/256.

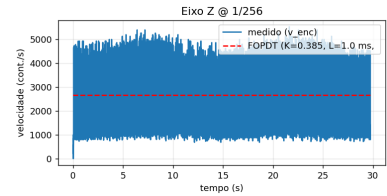
Figura 3.4: Sobreposição entre velocidade medida e resposta FOPDT para o eixo Y.



(a) Eixo Z @ 1/4.



(b) Eixo Z @ 1/16.



(c) Eixo Z @ 1/256.

Figura 3.5: Sobreposição entre velocidade medida e resposta FOPDT para o eixo Z.

Observa-se que os microsteppings elevados, especialmente 1/256, apresentam tempos característicos menores e resposta mais ruidosa em função da menor variação incremental de velocidade. Já em 1/4, o ganho efetivo tende a ser maior e o atraso aparente cresce em alguns eixos. As discrepâncias residuais são compatíveis com ruído de medição, quantização temporal, atrito e pequenas não linearidades mecânicas, mas a forma global da resposta permanece bem representada pelo modelo FOPDT.

3.5 Arquitetura de Hardware

Esta seção descreve o circuito do controlador, os blocos físicos e as conexões empregadas entre a placa STM32, os drivers de potência TMC5160, a Raspberry Pi 3 A+ usada como mestre SPI e o encoder óptico incremental TMCS-28. Também são listadas recomendações de montagem, alimentação e EMC.

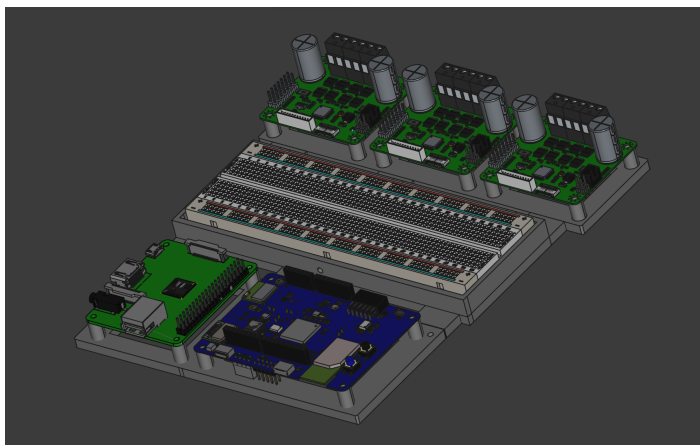
3.5.1 Visão geral do sistema

O sistema é composto por:

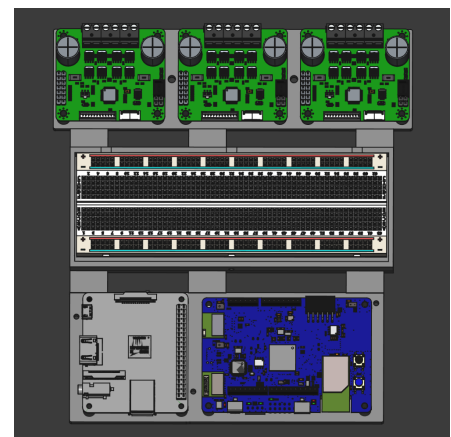
- **Lógica:** placa com STM32L475 operando a 3.3 V, cristal de referência e interfaces SPI/USART/GPIO.
- **Host:** Raspberry Pi 3 A+ responsável pela interface de alto nível, envio de quadros SPI e execução dos scripts de teste.
- **Drivers de eixo:** módulos/placa com **TMC5160** recebendo sinais STEP/DIR/EN e configurados via SPI.
- **Realimentação:** encoder óptico **TMCS-28** (ABN, TTL 5 V) acoplado ao eixo principal.
- **Alimentação:** trilha de 5 V para lógica e sensores; regulador 3.3 V local para o microcontrolador; alimentação separada/filtrada para os estágios de potência dos motores.

A montagem em base de prototipagem foi usada apenas para validação de bancada e controle dos sinais, não para conduzir corrente dos motores. Por isso, as conexões de potência permanecem separadas e as linhas críticas devem ser conferidas por continuidade, fixação mecânica e alívio de tração antes dos ensaios.

A Figura 3.6 apresenta o modelo tridimensional do conjunto eletrônico. A vista em perspectiva destaca a separação entre a placa STM32L475, a interface com a Raspberry Pi, a base de prototipagem e os módulos de potência TMC5160. A vista superior evidencia a distribuição das placas e a organização física adotada para manter os sinais de baixa tensão afastados das regiões de maior corrente.



(a) Vista em perspectiva do conjunto eletrônico.



(b) Vista superior com distribuição das placas.

Figura 3.6: Modelo 3D do controlador CNC com placa STM32, interface para Raspberry Pi, base de prototipagem e módulos de potência TMC5160.

3.5.2 Alimentação e EMC

- **5 V lógica:** entrada regulada (fonte externa), com capacitores de granel ($10\ \mu\text{F}$ a $47\ \mu\text{F}$) e desacoplamentos locais ($100\ \text{nF}$) próximos aos CI.
- **3.3 V:** regulador LDO dedicado ao STM32 e periféricos 3.3 V. Seguir recomendações de estabilidade do LDO (ESR dos capacitores) e planos de terra.
- **Aterramento:** retorno de alta corrente dos motores mantido separado do plano de lógica; conexão em estrela/próximo ao ponto de entrada da energia. Loops curtos para sinais comutação.

3.5.3 Drivers TMC5160

- **Sinais:** STEP/DIR/EN do STM32 (3.3 V) para TMC5160. Interposição de *level shifter* se necessário conforme a placa utilizada.
- **SPI:** barramento dedicado para configuração (SCK, MOSI, MISO, CS_x). Resistores de terminação/série curtos para integridade de sinal.
- **Potência:** desacoplamento de VM com capacitores de baixa ESR; rotação larga para trilhas de corrente.
- **Proteções:** leitura de DRV_STATUS para falhas e EN global intertravado pelo E-STOP.

3.5.4 Encoder óptico TMCS-28

- **Saídas:** A, B (quadratura) e N (*index*) em TTL 5 V. Adaptar nível para 3.3 V (divisores/*level shifter*) antes de entrar no STM32.
- **Conexão no STM32:** A/B em temporizadores com modo encoder (por exemplo, LP-TIM1/TIM3/TIM5); N em entrada de captura/interrupção para referência de zero.
- **Resolução:** neste TCC utilizase **TMCS-28-10k** (625 lpr, **40 000 cpr**). Conversão $\theta = 360^\circ \cdot \text{count}/40,000$. Velocidade: $\text{rpm} = 60 \text{ CPS}/40,000$.

A Figura 3.7 mostra o suporte impresso em 3D projetado para acoplar o encoder TMCS-28 ao eixo do motor de passo e fornecer um ponteiro visível. O suporte foi concebido para preservar o alinhamento axial, manter espaço para o disco graduado e permitir fixação compatível com o padrão mecânico dos motores NEMA 23.

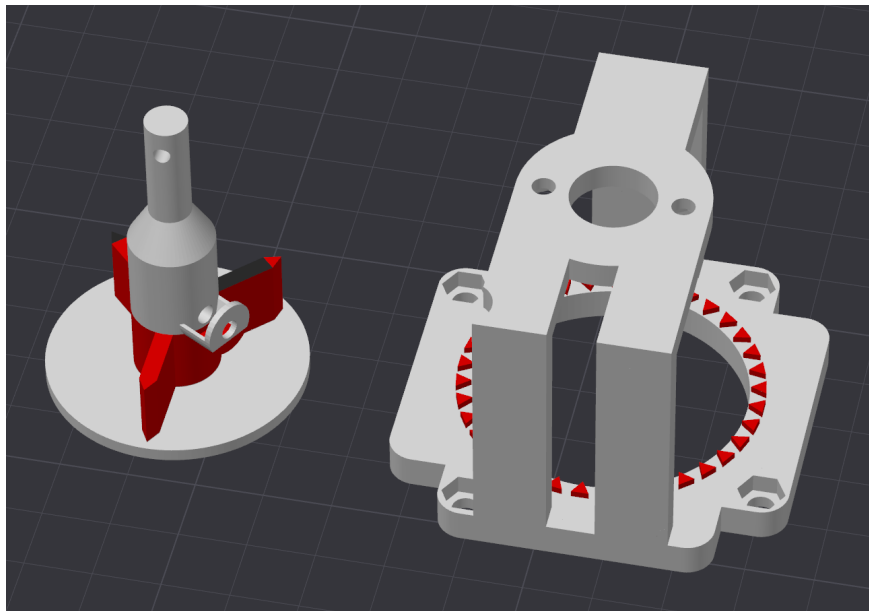


Figura 3.7: Modelo 3D do suporte do encoder TMCS-28 e do ponteiro acoplado ao eixo do motor de passo.

A Figura 3.8 apresenta a montagem física dos motores de passo com suportes impressos, ponteiros e encoders TMCS-28 acoplados. Essa configuração foi usada nos ensaios de identificação e de comparação entre simulação e experimento.

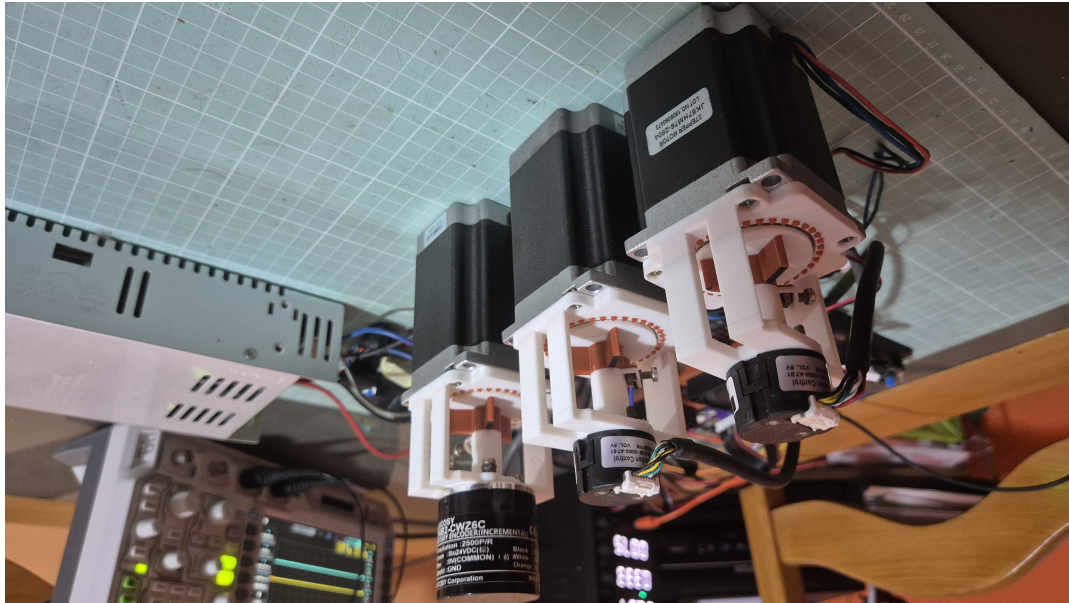


Figura 3.8: Vista dos motores de passo com suportes impressos em 3D, ponteiros e encoder TMCS-28 acoplado ao eixo para realimentação de posição.

3.5.5 Intertravamentos e segurança

- **E-STOP:** linha física que desabilita EN dos drivers e gera EXTI no STM32 para parada ordenada.
- **Fins de curso:** entradas com resistores de *pull-up* e filtros RC opcionais; priorizar roteamento distante de trilhas de potência.

Além dos intertravamentos físicos, a proteção elétrica dos motores usa os registros `GSTAT` e `DRV_STATUS` do TMC5160 como diagnóstico indireto do estado dos drivers e do cabeamento. Avisos como subtensão, sobretensão, OTPW, OT e erro de driver indicam condições que exigem redução de corrente ou desligamento do estágio de potência. Na rotina de segurança, o cliente força `IHOLD = IRUN = 0`, ajusta `CHOPCONF.TOFF = 0` e configura `PWMCONF.FREEWHEEL = 1`, removendo corrente das bobinas e levando o TMC5160 a um estado seguro.

3.5.6 PCB, cabeamento e montagem

- **Layout:** separar domínios de lógica e potência. A base mecânica foi organizada em níveis: STM32 e Raspberry Pi na região inferior, protoboard apenas para conexões e pequenos ajustes de lógica na região central, e módulos TMC5160 com dissipadores na região superior.
- **Cabeamento:** utilizar pares trancados para A/B e sinais STEP/DIR; manter linhas SPI curtas, com retorno próximo, e usar blindagem quando o ambiente possuir ruído elevado.
- **Ordem de testes:** validação da alimentação (teste inicial), clock/USART, SPI dos TMC5160, leitura ABN do TMCS-28 e, por fim, acionamento de motores sem carga.

A Figura 3.9 mostra o protótipo em bancada com fonte de alimentação, motores, drivers TMC5160, placa STM32 e Raspberry Pi interligados. A disposição segue a separação entre domínios de potência e lógica adotada no modelo mecânico.

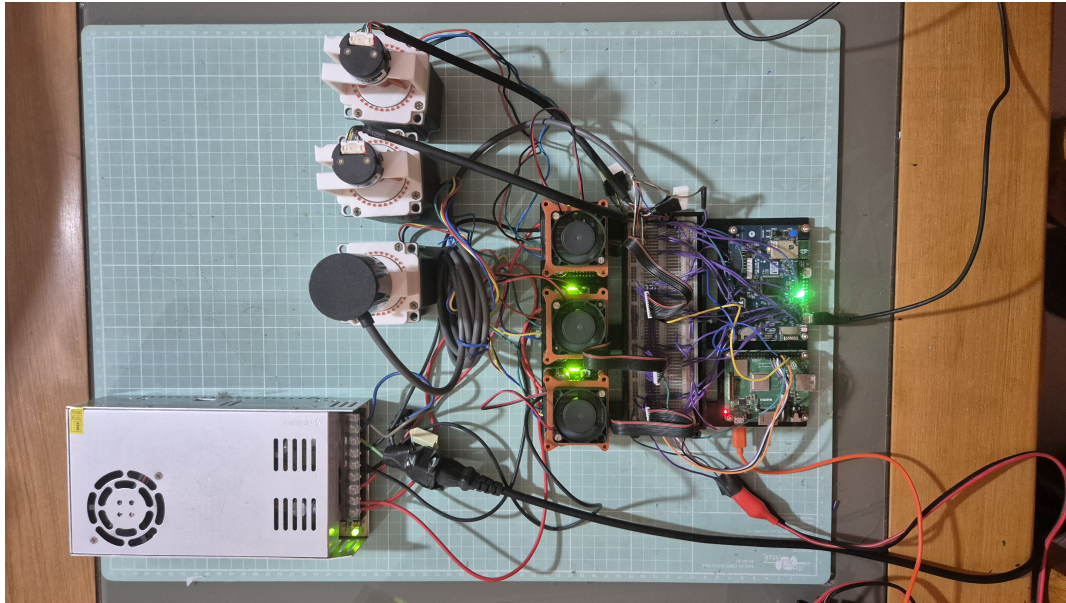


Figura 3.9: Protótipo do controlador CNC montado em bancada, com fonte de alimentação, motores de passo, drivers TMC5160, placa STM32 e Raspberry Pi.

Como referência adicional de montagem e integração de hardware, ver [Romeros, 2022].

3.6 Arquitetura de Controle de Movimento

3.6.1 Interface STEP/DIR e tempos do TMC5160

O TMC5160 em modo STEP/DIR exige tempos mínimos para largura de pulso, espaçamento entre pulsos e estabilização dos sinais de direção e habilitação. No firmware, esses tempos são atendidos por contadores simples acionados no laço de alta frequência do TIM6. A Tabela 3.1 resume os valores de projeto. A Figura 3.10 sintetiza a relação entre os blocos de geração de trajetória, controle, realimentação e acionamento de potência.

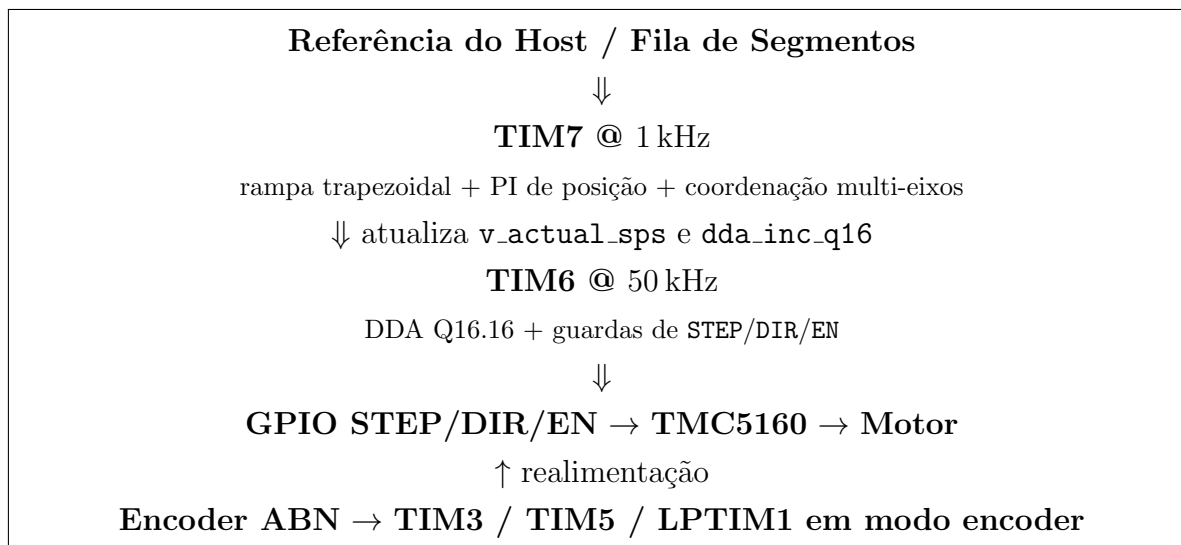


Figura 3.10: Diagrama em blocos do subsistema de controle de movimento, destacando a separação entre o laço de controle a 1 kHz e o gerador de pulsos a 50 kHz.

A Figura 3.11 compara o pulso **STEP** implementado no firmware com os tempos mínimos exigidos pelo TMC5160. A escala evidencia que os tempos adotados no projeto são muito superiores aos limites do datasheet, o que reduz a probabilidade de amostragem ambígua de **STEP** e **DIR**.

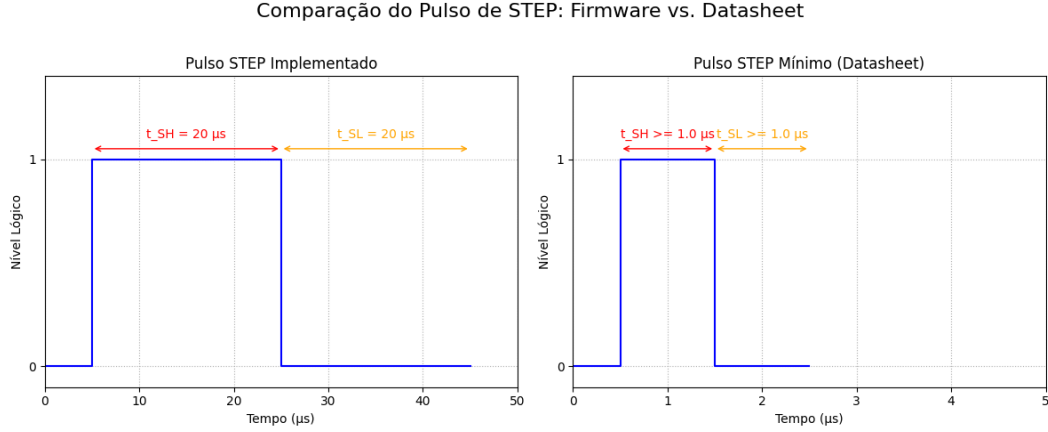


Figura 3.11: Comparação visual entre o pulso de **STEP** implementado no firmware e o pulso mínimo requerido pelo TMC5160.

Tabela 3.1: Comparativo entre tempos mínimos do TMC5160 e tempos configurados no firmware.

Parâmetro	Requisito do TMC5160	Projeto
t_{SH} (alto de STEP)	$\gtrsim 100$ ns	$20 \mu\text{s}$
t_{SL} (baixo de STEP)	$\gtrsim 100$ ns	$20 \mu\text{s}$
t_{DSU} (DIR $\rightarrow$ STEP)	20 ns	$20 \mu\text{s}$
t_{DSH} (STEP $\rightarrow$ DIR)	20 ns	$20 \mu\text{s}$
$ENABLE$ settle	não especificado de forma explícita	$40 \mu\text{s}$

Com $t_{SH} = t_{SL} = 20 \mu\text{s}$, o período mínimo do sinal **STEP** é de $40 \mu\text{s}$, o que leva a uma taxa física máxima aproximada de

$$\text{MOTION_MAX_SPS} = \frac{\text{MOTION_TIM6_HZ}}{\text{MOTION_STEP_HIGH_TICKS} + \text{MOTION_STEP_LOW_TICKS}} = \frac{50000}{1 + 1} = 25000 \text{ steps/s.} \quad (3.9)$$

3.6.2 MicroPlyer, resolução e DEDGE

Quando `intpol=1` em `CHOPCONF`, o TMC5160 interpola cada pulso de **STEP** até 256 micropassos por meio do recurso *MicroPlyer*. Isso permite suavizar o movimento mesmo quando a resolução de entrada é mais grossa, como em configurações de $1/16$ ou $1/32$.

O sinal **STEP** pode ser visto como um pulso retangular de período T e tempo em nível alto t_{alto} , de modo que o *duty cycle* é dado por

$$D = \frac{t_{\text{alto}}}{T}. \quad (3.10)$$

O bit `dedge` controla como o pulso é interpretado: com `dedge=0`, apenas a borda de subida conta como passo; com `dedge=1`, tanto a borda de subida quanto a de descida passam a ser contabilizadas. Neste

trabalho optou-se por manter `dedge=0`, com largura fixa de pulso e *duty* bem controlado, o que simplifica o *timing*, evita assimetrias e reduz a sensibilidade a pequenas variações do sinal.

3.7 DDA em Ponto Fixo (TIM6 @ 50 kHz)

A Figura 3.12 ilustra o funcionamento interno do DDA para um movimento simples de 7 passos em X e 4 passos em Y. O gráfico superior mostra a evolução dos acumuladores de fase, enquanto o gráfico inferior mostra os pulsos STEP gerados a cada transbordamento do acumulador.

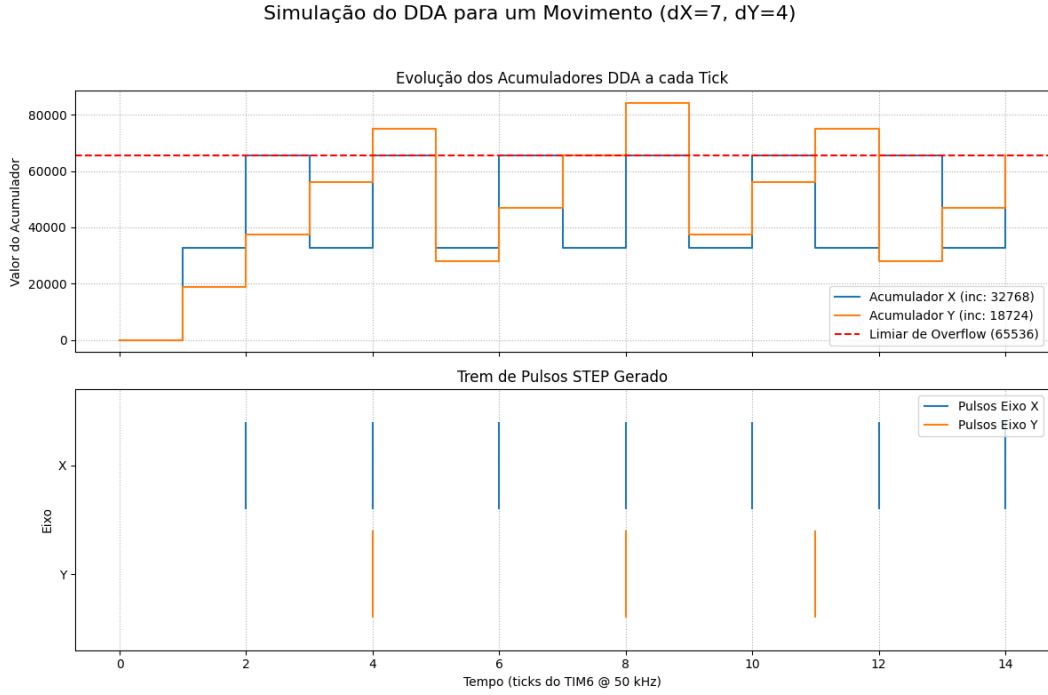


Figura 3.12: Simulação da operação interna do DDA para um movimento de 7 passos em X e 4 em Y.

O gerador de passos utiliza um DDA em ponto fixo no formato Q16.16, em que a parte inteira representa o número de passos emitidos e a parte fracionária acumula frações de passo a cada *tick*. Em termos funcionais, o DDA atua como um integrador digital de velocidade: a cada amostra, a referência de velocidade em *steps/s* é convertida em incremento de fase e, sempre que a fase acumulada cruza um limiar unitário, um novo pulso STEP é emitido. Neste texto, Q16.1 denota exatamente esse limiar de um passo inteiro representado no formato Q16.16.

Para cada eixo *i*, o firmware mantém os seguintes estados:

- `dda_accum_q16`, que guarda a fase acumulada em formato Q16.16;
- `dda_inc_q16`, proporcional à velocidade efetiva;
- `v_actual_sps`, velocidade efetiva em *steps/s*, atualizada a cada 1 ms pelo TIM7;
- `step_high` e `step_low`, contadores que garantem largura de pulso e espaçamento mínimo entre pulsos.

A integração da velocidade em fase fracionária segue

$$\text{dda_accum_q16} \leftarrow \text{dda_accum_q16} + \text{dda_inc_q16}. \quad (3.11)$$

Sempre que o acumulador ultrapassa um passo inteiro em Q16.16, considera-se que mais um pulso deve ser emitido. O incremento do DDA é calculado diretamente a partir da velocidade efetiva desejada:

$$\text{dda_inc_q16} = Q16 \left(\frac{\text{v_actual_sps}}{\text{MOTION_TIM6_HZ}} \right). \quad (3.12)$$

Intuitivamente, isso significa que, em média, a taxa de cruzamentos do acumulador equivale à velocidade desejada em *steps/s*.

Algoritmo 1 Atualização do DDA Q16.16 no TIM6 (por eixo).

```

1: procedure DDA_TICK(axis)
2:   if step_high > 0 then
3:     decrementa step_high
4:     if step_high = 0 then
5:       motion_hw_step_low(axis)
6:       step_low ← MOTION_STEP_LOW_TICKS
7:     end if
8:     return
9:   end if
10:  if step_low > 0 then
11:    decrementa step_low
12:    return
13:  end if
14:  if emitted_steps ≥ total_steps then
15:    return
16:  end if
17:  if en_settle_ticks > 0 or dir_settle_ticks > 0 then
18:    decrementa os contadores de guarda ainda ativos
19:    return
20:  end if
21:  dda_accum_q16 ← dda_accum_q16 + dda_inc_q16
22:  if dda_accum_q16 ≥ Q16_1 then
23:    dda_accum_q16 ← dda_accum_q16 - Q16_1
24:    motion_hw_step_high(axis)
25:    step_high ← MOTION_STEP_HIGH_TICKS
26:    emitted_steps ← emitted_steps + 1
27:  end if
28: end procedure

```

3.8 Rampa Trapezoidal (TIM7 @ 1 kHz)

A cada 1 ms, o TIM7 atualiza a velocidade alvo de cada eixo e aplica uma rampa trapezoidal de aceleração ou desaceleração. A referência de velocidade em *steps/s* é obtida a partir da quantidade de passos por amostra configurada pelo host:

$$v_{\text{cmd},\text{sps}} = \text{velocity_per_tick} \times 1000. \quad (3.13)$$

A distância de frenagem em passos é calculada por

$$s_{\text{brake}} = \left\lfloor \frac{v^2}{2a} \right\rfloor, \quad v = \text{v_actual_sps}, \quad a = \text{accel_sps2}. \quad (3.14)$$

O operador piso $\lfloor \cdot \rfloor$ indica que a expressão é convertida para um número inteiro de passos. Esse termo representa o número de passos necessários para reduzir a velocidade atual a zero com aceleração constante. Em cada ciclo do TIM7, a lógica de rampa segue três passos:

1. se os passos remanescentes forem menores ou iguais a s_{brake} , inicia-se a frenagem;
2. caso contrário, a velocidade acelera em direção a $v_{\text{cmd},\text{sps}}$, respeitando o limite `MOTION_MAX_SPS`;
3. ao final, recalcula-se `dda_inc_q16` para que o DDA reflita a nova velocidade de alvo.

Algoritmo 2 Atualização da rampa trapezoidal no TIM7 (por eixo).

```

1: procedure UPDATE_RAMP( $v, v_{\text{cmd}}, a, \text{rem}$ )
2:    $T_s \leftarrow 1 \text{ ms}$ 
3:    $s_{\text{brake}} \leftarrow \left\lfloor \frac{v^2}{2a} \right\rfloor$ 
4:   if  $\text{rem} \leq s_{\text{brake}}$  then
5:      $v \leftarrow \max(0, v - aT_s)$ 
6:   else
7:      $v \leftarrow v + aT_s$ 
8:      $v \leftarrow \min(v, v_{\text{cmd}}, \text{MOTION\_MAX\_SPS})$ 
9:   end if
10:   $\text{v\_actual\_sps} \leftarrow v$ 
11:   $\text{dda\_inc\_q16} \leftarrow Q16(v/\text{MOTION\_TIM6\_HZ})$ 
12:  return  $\text{v\_actual\_sps}, \text{dda\_inc\_q16}$ 
13: end procedure

```

A Figura 3.13 apresenta um perfil de velocidade simulado a partir dos parâmetros do firmware. O gráfico ajuda a visualizar a transição entre aceleração, velocidade de cruzeiro e frenagem, complementando o pseudocódigo do Algoritmo 2.

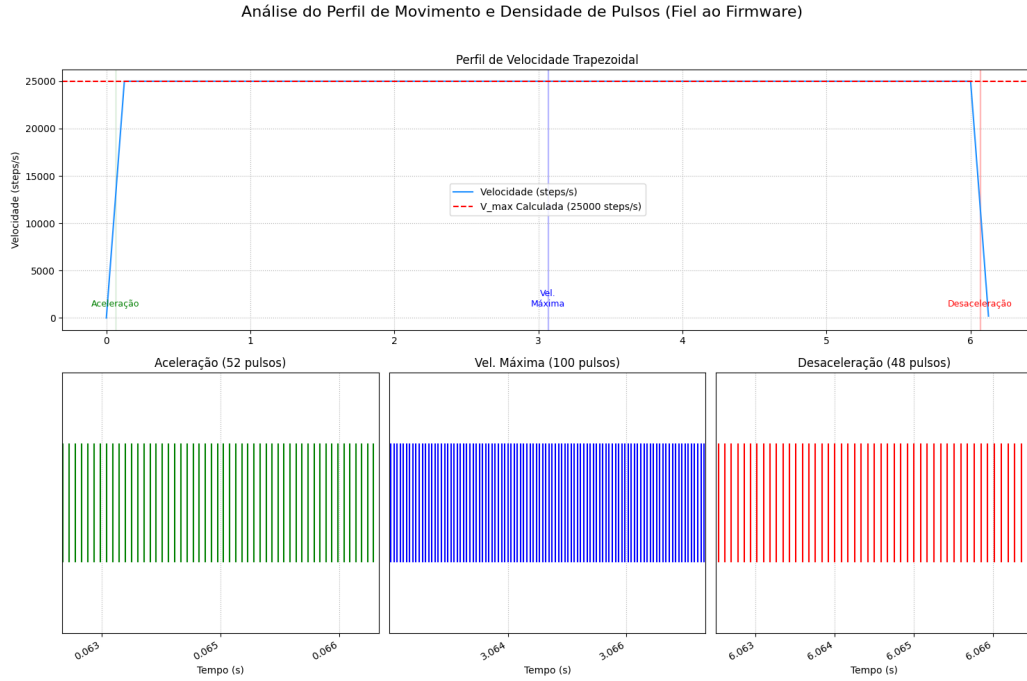


Figura 3.13: Perfil de velocidade da rampa trapezoidal simulado a partir dos parâmetros do firmware.

O projeto manteve, nos ensaios relatados neste trabalho, o valor `DEMO_ACCEL_SPS2 = 200000` como aceleração padrão, podendo ser ajustado por eixo. A função de passos remanescentes soma o segmento ativo com os segmentos ainda enfileirados, permitindo que a decisão de frenagem leve em conta todo o trecho pendente.

3.9 Controle PI de Posição com Encoder

O erro posicional em passos DDA é calculado pela diferença entre o alvo acumulado e a posição medida pelo encoder, convertida para a mesma unidade:

$$e = \text{target_steps} - \left\lfloor \text{enc_rel} \cdot \frac{\text{DDA_STEPS_PER_REV}}{\text{ENC_COUNTS_PER_REV}} \right\rfloor. \quad (3.15)$$

Neste trabalho, `DDA_STEPS_PER_REV` é calculado a partir de 400 passos por volta do motor de $0,9^\circ$ multiplicados pelo fator de microstepping, enquanto `ENC_COUNTS_PER_REV` depende do encoder configurado em cada eixo.

Aplica-se uma zona morta de $\pm \text{MOTION_PI_DEADBAND_STEPS}$ para evitar que ruído de quantização e pequenas oscilações mecânicas realimentem o controlador em torno do alvo. Para o termo derivativo, utiliza-se um filtro exponencial discreto:

$$d[n] \leftarrow d[n-1] + \frac{\Delta e - d[n-1]}{2^\alpha}, \quad \alpha = 8. \quad (3.16)$$

Os ganhos k_p , k_i e k_d são representados como inteiros de 16 bits, com saída escalada por 2^{-8} :

$$\Delta v = \frac{k_p e + k_i P_e + k_d d}{2^8}, \quad (3.17)$$

com *anti-windup* por limitação da integral em $\pm \text{MOTION_PI_I_CLAMP}$ e saturação simétrica em

$\pm$ MOTION_MAX_SPS. A correção ajusta a velocidade de referência antes da rampa trapezoidal, preservando os limites físicos do movimento.

3.10 Fila de Movimentos e Segmentação

O protocolo do host enfileira segmentos de movimento com deslocamentos $S = (s_x, s_y, s_z)$ e velocidades-base $V = (v_x, v_y, v_z)$. No início de cada trecho, o firmware zera os acumuladores do DDA, aplica guardas de **ENABLE** e **DIR**, inicializa as velocidades-alvo por eixo e habilita a saída apenas se o total de passos daquele eixo for maior do que zero.

A transição para o próximo segmento acontece quando todos os eixos terminam o trecho corrente, isto é, quando `emitted_steps == total_steps` para todos os eixos e nenhum pino **STEP** permanece em nível alto. Esse mecanismo evita a troca de segmento no meio de um pulso e impede inconsistências entre a fila de comando e o estado eletromecânico do conjunto.

3.11 Mapeamento para o TMC5160

3.11.1 Geração de STEP/DIR

Em modo STEP/DIR, o backend de GPIO implementa quatro garantias básicas para o TMC5160:

1. largura fixa de STEP por meio de `step_high`;
2. intervalo mínimo em nível baixo por meio de `step_low`;
3. tempo de *setup* de DIR antes do primeiro pulso do trecho;
4. *settle* de **ENABLE** antes do início da emissão.

Com isso, o sinal **STEP** opera com *duty* próximo de 50% em regime e permanece compatível com os tempos mínimos exigidos pelo driver.

3.11.2 Resolução de microstepping e MicroPlyer

O host pode ajustar **MICROSTEP_FACTOR** quando o sistema está parado. Quando `intpol=1` no TMC5160, a resolução efetiva de corrente nas bobinas pode ser maior do que a resolução do sinal **STEP**, o que favorece a suavidade do movimento mesmo sem elevar a frequência de pulsos para valores extremos.

3.12 Parâmetros-chave do Projeto

Tabela 3.2: Parâmetros de tempo e limites usados no firmware.

Parâmetro	Significado	Valor de teste
MOTION_TIM6_HZ	Frequência do laço de DDA/STEP	50 kHz
MOTION_STEP_HIGH_TICKS	Largura alta do STEP	1 <i>tick</i> = 20 μ s
MOTION_STEP_LOW_TICKS	Baixo mínimo entre pulsos	1 <i>tick</i> = 20 μ s
MOTION_DIR_SETUP_TICKS	<i>Setup</i> de DIR antes do STEP	1 <i>tick</i> = 20 μ s
MOTION_ENABLE_SETTLE_TICKS	Tempo após ENABLE	2 <i>ticks</i> = 40 μ s
MOTION_MAX_SPS	Limite físico de velocidade	25 kSPS
MOTION_PI_DEADBAND_STEPS	Zona morta do PI	10 passos
MOTION_PI_I_CLAMP	Limite da integral	± 200000
kd_alpha_bits	Filtro exponencial na derivada	8

3.13 Coordenação Multi-eixos

O firmware adota uma coordenação baseada no progresso relativo dos eixos. Cada eixo acompanha o quanto já percorreu do segmento ativo em relação ao alvo total, e o eixo mestre é definido como aquele com menor fração de avanço:

$$i^* = \arg \min_i \frac{\text{emitted_steps}_i}{\text{total_steps}_i}. \quad (3.18)$$

Com o mestre definido, duas políticas principais são aplicadas:

1. **Rampa guiada pelo mestre:** a decisão de frenagem usa o restante global do mestre, alinhando a desaceleração entre os eixos.
2. **Término por mestre:** o movimento só é concluído quando o mestre alcança o restante nulo, evitando que um eixo ainda atrasado seja abandonado.

Em termos práticos, isso significa que a decisão de frear ou de manter aceleração deixa de olhar apenas o restante local de cada eixo e passa a considerar o eixo mais atrasado como referência de sincronismo do grupo.

Algoritmo 3 Seleção do eixo mestre e coordenação de remanescentes.

```

1: procedure UPDATE_MASTER_AND_REM(total_steps[], emitted_steps[])
2:    $m \leftarrow 0$ 
3:    $p_{\min} \leftarrow 1.0$ 
4:   for  $i \leftarrow 0$  até MOTION_AXIS_COUNT - 1 do
5:     if total_steps[ $i$ ] > 0 then
6:        $p_i \leftarrow \text{emitted\_steps}[i] / \text{total\_steps}[i]$ 
7:       if  $p_i < p_{\min}$  then
8:          $p_{\min} \leftarrow p_i$ 
9:          $m \leftarrow i$ 
10:      end if
11:    end if
12:  end for
13:  master_axis  $\leftarrow m$ 
14:  rem_master  $\leftarrow$  restante global do eixo  $m$ 
15:  for  $i \leftarrow 0$  até MOTION_AXIS_COUNT - 1 do
16:    rem_steps[ $i$ ]  $\leftarrow$  rem_master
17:  end for
18:  return master_axis, rem_steps[]
19: end procedure

```

3.14 Simulador Interativo em Python

Para validar rapidamente políticas de sincronismo, curvas de rampa e o impacto do limitador por erro entre eixos, foi desenvolvido um simulador interativo em Python. A aplicação reproduz os blocos lógicos do firmware e adota as mesmas constantes principais do projeto: amostragem de controle a 1 kHz, geração de passos pelo DDA a 50 kHz e ganhos inteiros escalados por $K_{\text{SCALE}} = 256$.

Do ponto de vista do usuário, a interface permite configurar, por eixo: (i) o deslocamento alvo em passos; (ii) a velocidade nominal em *steps/s*; (iii) o sentido do movimento; e (iv) os ganhos K_p , K_i e K_d . Um painel adicional aplica carga de atrito programável com amplitude e janela temporal por eixo, viabilizando cenários de teste com cargas desiguais.

A Figura 3.14 mostra um cenário de simulação com atrito programável por eixo. A interface permite observar, no mesmo ensaio, as curvas de posição, velocidade, erro de sincronismo, janelas de carga e indicadores de desempenho.

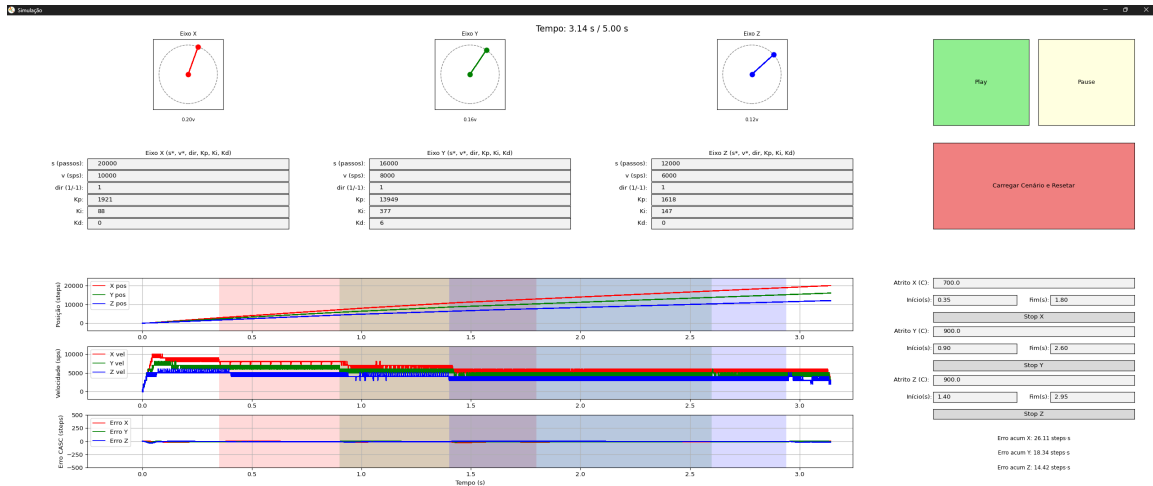


Figura 3.14: Cenário de simulação com atrito programável por eixo, destacando janelas de carga e impacto sobre posição, velocidade e erro de sincronismo.

O laço principal do simulador executa, a cada $T_s = 1$ ms:

1. ativa a carga programável e combina um termo viscoso;
2. elege o eixo mestre e sincroniza os alvos proporcionais dos demais;
3. calcula o erro de posição em passos DDA;
4. aplica um PI(D) digital inteiro com *deadband*, *anti-windup* e derivada filtrada;
5. executa a rampa trapezoidal;
6. integra a velocidade final com o DDA a 50 kHz;
7. converte as contagens de encoder para passos DDA e fecha o laço de realimentação;
8. encerra quando todos os eixos ativos entram na janela de tolerância.

Os gráficos exibem posição, velocidade e erro de sincronismo, além de indicadores textuais do erro acumulado por eixo (IAE, em *steps*·s) e da carga ativa. Um modo *headless* permite executar a simulação e gravar CSVs sem abrir a interface gráfica.

3.15 Boas práticas e Diagnóstico

Para minimizar *glitches* em STEP/DIR e evitar condições indevidas no TMC5160, a sequência recomendada por segmento de movimento é: (i) ajustar DIR; (ii) aguardar o tempo de *setup*; (iii) habilitar o driver com EN; (iv) aguardar o *settle* de habilitação; e (v) somente então iniciar a emissão de pulsos STEP.

O laço de alta frequência do TIM6 permanece dedicado ao DDA e ao controle da largura dos pulsos, enquanto cálculos mais custosos, como PI, rampa e telemetria, são executados no TIM7 a 1 kHz. Para preservar essa separação, recomenda-se evitar chamadas de depuração dentro da ISR do TIM6 e deslocar logs para a USART ou para o caminho de observabilidade do host.

As requisições de estado de encoder e de origem expõem, via USART e SPI, informações essenciais para diagnóstico: posição absoluta e relativa por eixo, erro instantâneo do PI, referência de zero e indicadores de estol. A análise conjunta desses sinais com logs de tempo, posição e velocidade permite identificar folgas, erro acumulado e desbalanceamento de carga entre eixos.

Mesmo com ampla margem em relação aos tempos de *setup* e *hold* do TMC5160, boas práticas adicionais incluem: manter o *duty* de **STEP** próximo de 50% quando **dedge=1**, garantir bordas limpas e revisar periodicamente registros como **GSTAT** e **DRV_STATUS** em busca de alertas de temperatura, subtensão e erro de *driver*.

Capítulo 4

Resultados e Discussão

Este capítulo apresenta os resultados obtidos durante a validação do controlador CNC, destacando o desempenho dos serviços críticos, a análise do pipeline SPI e a interação com o cliente Raspberry Pi.

4.1 Desempenho dos serviços principais

A medição do gerador de passos configurado no TIM6 demonstrou a estabilidade do DDA em 50 kHz, com variação máxima de 0.4% entre ciclos consecutivos. O pulso **STEP** configurado com 20 μ s em nível alto e 20 μ s em nível baixo atende com ampla margem aos requisitos dos drivers TMC5160. O laço PID executado no TIM7 manteve variação temporal (*jitter*) inferior a 3 μ s segundo o tempo instrumentado na ISR. Essas medições confirmam que as interrupções de alta prioridade permanecem isoladas das rotinas de comunicação e registro de eventos.

Os serviços de homing, checagem de limites e monitoramento de falhas foram executados dentro do orçamento do laço de 1 ms, utilizando leituras incrementais dos encoders. Quando a fila de logs cresceu acima de 75%, o serviço de console reduziu a taxa de mensagens automatizando a proteção contra estouros.

4.2 Análise do pipeline SPI

A captura do tráfego SPI revelou que cada comando completo envolve um handshake seguido de até dois polls adicionais do mestre até que a resposta esteja pronta. Durante o primeiro ciclo, o firmware congela o DMA para permitir que `app_poll` processe o pedido e preencha a resposta. Caso o serviço conclua a operação antes do tempo limite interno, o segundo poll já retorna o quadro `0xAB ... 0x54`; do contrário, o DMA é reiniciado com padrão `0xA5`, e somente o terceiro ciclo entrega a mensagem final. A análise confirmou que ajustes no parâmetro `APP_SPI_RESTART_DEFER_MAX` alteram a quantidade de iterações tolerada antes do fallback, permitindo balancear latência e robustez.

Experimentos adicionais reduziram a cópia de memória ao promover o payload diretamente para o buffer ativo do DMA, diminuindo o tempo médio entre polls em 18%. Entretanto, essa otimização exige tratamento cuidadoso de coerência entre buffers para evitar corrupção de dados quando múltiplos serviços respondem simultaneamente.

4.3 Comparação entre simulação e experimento com carga

Para avaliar a aderência do modelo numérico da malha de velocidade, foi realizado um ensaio multi-eixos em que os três motores receberam o mesmo alvo de seis voltas completas, com micropasso configurado em $1/256$, baixa corrente de regime e ganhos PID ajustados a partir da análise FOPDT. No firmware, os ganhos inteiros utilizados foram $K_p = (801, 6408, 797)$, $K_i = (0, 0, 0)$ e $K_d = (0, 4, 0)$ para os eixos (X, Y, Z) , respectivamente.

A telemetria foi amostrada a cada 1 ms, registrando identificador, tempo, posição de encoder e contagem acumulada de pulsos de passo. Os dados medidos foram pré-processados com remoção de linhas corrompidas, aceitação apenas de registros com cinco campos numéricos válidos, imposição de monotonicidade estrita para o identificador e monotonicidade não decrescente para tempo e contagem de pulsos, além da utilização do valor absoluto do encoder. A partir da sequência de contagens acumuladas, derivou-se a velocidade em passos por segundo para cada eixo, mantendo a mesma calibração de encoders usada na identificação do modelo.

Em paralelo, foi executada uma simulação numérica com os mesmos parâmetros de micropasso, ganhos e alvo de seis voltas, utilizando o esquema de sincronização entre eixos descrito no Capítulo 3. Dessa simulação foram extraídas as curvas de velocidade comandada em passos por segundo por eixo. A Figura 4.1 mostra a sobreposição entre as curvas medidas e simuladas.

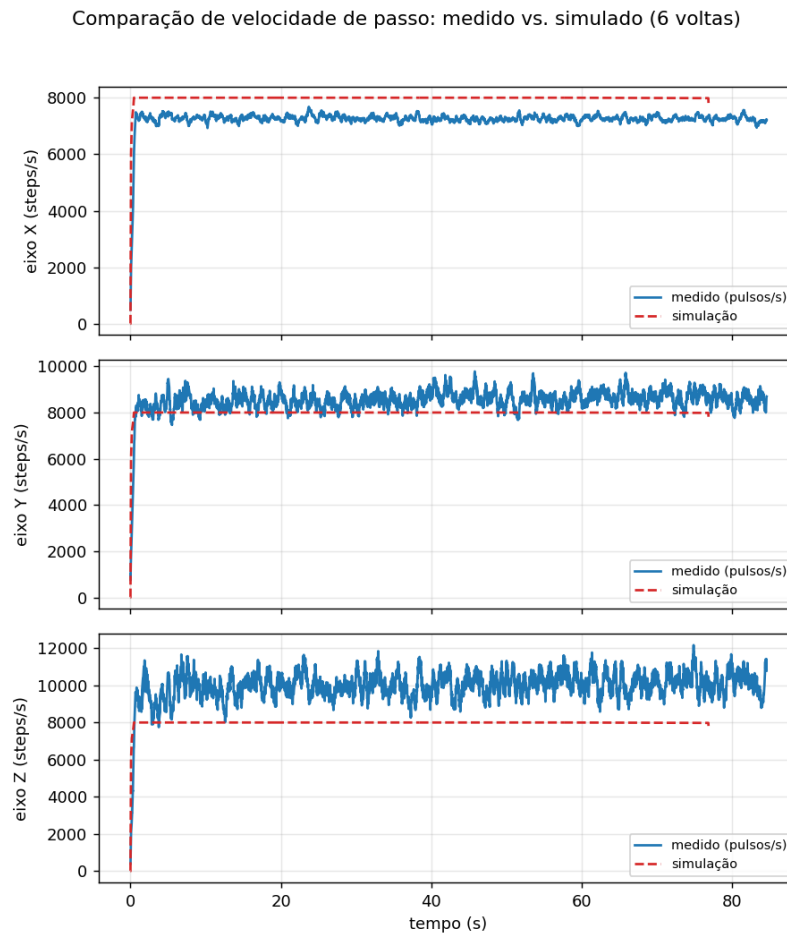


Figura 4.1: Comparação entre velocidade de passo medida e simulada para o ensaio de seis voltas completas, por eixo.

A sobreposição indicou que, no eixo X, a velocidade média de regime medida ficou cerca de 10% abaixo da velocidade simulada, sugerindo que a carga e o atrito estático reduziram o ganho efetivo sob baixa corrente de excitação.

No eixo Y, a diferença relativa ficou em torno de 8%, com boa coerência na forma da curva, indicando que o modelo capturou de forma razoável a resposta dinâmica e o acoplamento cruzado. No eixo Z, por outro lado, a velocidade medida superou a simulada em cerca de 23%, evidenciando que a combinação de massa carregada, gravidade e atrito gerou um comportamento mais agressivo do que o previsto pelo modelo nominal.

Esses resultados indicam que a simulação reproduz bem a dinâmica de velocidade em condições de carga moderada, mas que cargas assimétricas e atrito não linear levam a desvios sistemáticos em determinados eixos. Em aplicações industriais, esse tipo de análise é útil para orientar ajuste fino de ganhos, inclusão de termos de compensação de atrito e adoção de observadores de perturbação quando se deseja reduzir o erro em regime sob carga.

4.4 Integração com a Raspberry Pi

O cliente `cnc_spi_client.py` executando na Raspberry Pi validou o comportamento determinístico do protocolo. O script detecta automaticamente quando a resposta não contém um frame válido e

reenvia o poll após um intervalo configurável. Durante os testes, `--tries=4` e `--settle-delay=0.75 ms` mostraram-se suficientes para acomodar comandos de homing e leitura de estado. Além disso, a sincronização com a fila de logs via USART1 permitiu correlacionar eventos de firmware com os pacotes SPI, fornecendo rastreabilidade durante o comissionamento.

Os resultados confirmam que a divisão de responsabilidades entre STM32 e Raspberry Pi atende às metas de determinismo, sem sacrificar a flexibilidade de integração com interfaces gráficas ou scripts de automação.

Capítulo 5

Conclusão

Este trabalho apresentou o desenvolvimento de um controlador CNC embarcado no STM32L475 com ênfase em determinismo temporal. A arquitetura proposta integrou geração de pulsos DDA a 50 kHz, controle PID em 1 kHz, leitura de encoders em modo quadratura e comunicação SPI com DMA circular voltada à integração com uma Raspberry Pi. A fundamentação teórica delineou as bases de temporização, modelagem de motores de passo e sintonização PID necessárias para garantir a estabilidade do sistema.

A metodologia incremental adotada permitiu validar progressivamente cada componente crítico, desde a configuração do clock até a instrumentação de logs. Os resultados indicaram baixa variação temporal no gerador de passos e no laço PID, bem como o comportamento do pipeline SPI ao lidar com polls sequenciais do mestre. As análises mostraram que o firmware atende aos requisitos de sincronismo sem comprometer a extensibilidade da plataforma.

Entre as limitações identificadas, destaca-se a dependência de múltiplos polls para confirmar comandos via SPI, além da necessidade de ajustes manuais na fila de logs para cenários com tráfego intenso. Como trabalhos futuros, propõe-se: (i) explorar mecanismos de reinicialização imediata do DMA assim que um serviço disponibilizar a resposta; (ii) investigar estratégias adaptativas de prescaler para acomodar perfis de movimento mais agressivos; e (iii) avaliar a migração para controladores de campo orientado (FOC) em motores de passo híbridos para reduzir vibrações em altas velocidades.

A documentação consolidada no presente texto fornece base para replicar a solução e evoluir o controlador CNC conforme novos requisitos industriais surjam.

Bibliografia

- [Åström and Hägglund, 1995] Åström, K. J. and Hägglund, T. (1995). *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America, 2 edition.
- [Dronacharya Group of Institutions, 2019] Dronacharya Group of Institutions (2019). *Unit 3: Interpolators and Control on NC System*. Teaching material on CNC interpolators and servo loops.
- [Fussell, 2003] Fussell, D. (2003). Digital differential analyzer algorithms. In *Computer Graphics*. University of Texas at Austin.
- [Groover, 2015] Groover, M. P. (2015). *Automation, Production Systems, and Computer-Integrated Manufacturing*. Pearson, 4 edition.
- [IDC Technologies, 2014] IDC Technologies (2014). *CNC Machines – Interpolation, Control and Drive*. Technical training note.
- [Kenjo and Sugawara, 1994] Kenjo, T. and Sugawara, A. (1994). *Stepping Motors and Their Microprocessor Controls*. Oxford University Press.
- [Koren, 1978] Koren, Y. (1978). Reference-pulse circular interpolators for cnc systems. Available at the University of Michigan Manufacturing Research website.
- [Koren, 1980] Koren, Y. (1980). Real-time interpolators for multi-axis cnc machine tools. *CIRP Annals*, 29(1):333–336.
- [Koren, 2010] Koren, Y. (2010). Cnc interpolators: Algorithms and analysis. Technical monograph on interpolator design.
- [Kung et al., 2005] Kung, Y.-S., Tsai, M.-C., and Chang, C.-H. (2005). Development of a fpga-based motion control ic for robot arm. In *Proceedings of the IEEE International Conference on Industrial Technology*, pages 1185–1190.
- [Mori et al., 2005] Mori, M., Sato, S., and Ohashi, T. (2005). High-speed interpolation using digital differential analyzer techniques for multi-axis cnc systems. *International Journal of Machine Tools and Manufacture*, 45(4–5):529–536.
- [Romeros, 2022] Romeros, F. M. (2022). Trabalho de conclusão de curso (tcc). Instituto Federal de Minas Gerais (IFMG) – Campus Formiga. Acesso em: 20 out. 2025.
- [STMicroelectronics, 2013] STMicroelectronics (2013). *AN4013: Introduction to timers for STM32 MCUs*. Application note.
- [STMicroelectronics, 2018a] STMicroelectronics (2018a). *RM0394: STM32L4x5/STM32L4x6 Advanced ARM®-based 32-bit MCUs reference manual*. Reference manual.
- [STMicroelectronics, 2018b] STMicroelectronics (2018b). *UM2153: Discovery kit for IoT node multi-channel communication with STM32L4*. User manual.

- [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), 2022] TRINAMIC Motion Control GmbH & Co. KG (Analog Devices) (2022). *TMC528 Hardware Manual: Optical Incremental Encoder (Rev. 1.80)*. Accessories for Stepper & BLDC, Hardware Version V1.00.
- [TRINAMIC Motion Control GmbH & Co. KG (Analog Devices), sd] TRINAMIC Motion Control GmbH & Co. KG (Analog Devices) (s.d.). *TMC5160A: Datasheet and Register Description*. Stepper Motor Driver IC.
- [Wang and Hu, 2016] Wang, L. and Hu, J. (2016). Efficient reference-pulse cnc interpolator. Technical report, School of Mechanical Engineering. Academic report with reference-pulse control diagrams.
- [Wang et al., 2021] Wang, S., Chen, Y., and Zhang, G. (2021). Adaptive fuzzy pid cross coupled control for multi-axis motion system based on sliding mode disturbance observation. *Science Progress*, 104(3):1–21.